

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

PROJECT REPORT

VibeLearn: AI-Powered Personalized Learning and Adaptive Revision Platform

Submitted by

Abhijay Nagal

B.E. Computer Science and Engineering
5th Semester | Academic Year 2025–2026

Project Type: Independent Personal Software Engineering Project
Current Version: v2.0

July 2026

Declaration

I, Abhijay Nagal, a student of the Department of Computer Science and Engineering at Thapar Institute of Engineering and Technology, Patiala, hereby declare that this project report titled “VibeLearn: AI-Powered Personalised Learning and Adaptive Revision Platform: Version 2.0” has been independently prepared by me as part of a personal software engineering initiative undertaken as a direct continuation of the version-1 MVP documented in the preceding report.

I further declare that:

- This project and the accompanying report represent my original work and have not been submitted, either in part or in full, to any other institution or examination body for the award of any degree, diploma, or certificate.
- All external libraries, frameworks, APIs, and tools used in the course of this project have been duly acknowledged in the References and Appendix sections of this report.
- The architectural decisions, implementation strategies, and engineering tradeoffs documented herein reflect my own analysis and understanding of the problem domain, built upon the foundation established in Phase-I.
- Every feature described as implemented in this report corresponds to an actual, working component of the deployed application; no capability has been exaggerated or fabricated.
- The codebase, database schema, and API design described in this report were entirely conceived, designed, and implemented by me, with no academic dishonesty of any form.



Abhijay Nagal
B.E. Computer Science and Engineering
Thapar Institute of Engineering and Technology
Date: July 2026

Abstract

The Phase-I Minimum Viable Product (MVP) of VibeLearn established the core premise of the platform: transforming passive YouTube video consumption into structured, AI-generated notes, formative MCQ (Multiple Choice Question) assessment, and a basic adaptive revision loop. That report concluded with a clearly defined Phase-II roadmap, the most critical item of which was the replacement of the prototype's custom localStorage-based authentication with a cryptographically secure alternative. This report documents the completion of that roadmap and the substantial architectural maturation that followed from it.

Version 2.0 of VibeLearn migrates the entire identity layer to Supabase Authentication, including Google OAuth (Open Authorization), server-side session verification via `supabase.auth.getUser()`, and full removal of localStorage-based identity. Every session, video, and quiz record is now bound to an authenticated user, with ownership verified on every protected API (Application Programming Interface) route. Building on this authenticated foundation, this phase introduces an Intelligent Notes Cache and an Intelligent Quiz Cache, which together eliminate redundant transcript extraction, redundant LLM (Large Language Model) inference, and redundant Groq API token consumption when multiple users study the same YouTube video. The database schema was restructured around a global `yt_video_id`-keyed caching strategy, which required resolving several constraint and migration challenges encountered during development.

In parallel with these backend changes, the application underwent a comprehensive interface redesign: a semantic, theme-aware design system, a collapsible sidebar navigation pattern, dark mode support via next-themes, and redesigned quiz, revision, dashboard, and authentication pages. Dashboard analytics were re-implemented using PostgreSQL Remote Procedure Calls (RPCs) to move heavy aggregation work into the database layer, avoiding Vercel serverless execution timeouts.

This report documents the complete, current state of the VibeLearn platform across every chapter of the original Phase-I report, updated to reflect Version 2.0, alongside new chapters covering Authentication Architecture, the Intelligent Caching System, Session Ownership and Authorization, the Premium UI System, and Analytics Architecture. It is written from the perspective of the sole developer immediately following the completion of this second major development phase, and is intended to serve as both technical documentation and a reflective account of the engineering process.

Keywords: *Supabase Authentication, OAuth, Adaptive Learning, Intelligent Caching, PostgreSQL RPC, Next.js, Groq API, Llama 3.3, Row-Level Authorization, Dark Mode Design Systems, Full-Stack Development.*

Table of Contents

Contents

Declaration.....	1
Abstract.....	3
Table of Contents.....	4
Chapter 1 – Introduction.....	9
1.1 Project Overview	9
1.2 Motivation for Phase-II.....	9
1.3 Report Organisation	9
Chapter 2 – Background and Motivation	10
2.1 The YouTube-as-Education Phenomenon	10
2.2 The Forgetting Curve and Active Learning.....	10
2.3 Limitations of Existing Solutions	10
2.4 Developer's Learning Goals for Phase-II	10
Chapter 3 – Problem Statement.....	11
3.1 Core Problem (Restated for Phase-II).....	11
3.2 Problem Scope	11
3.3 Desired Outcome.....	11
Chapter 4 – Objectives.....	12
4.1 Primary Objectives (Phase-II).....	12
4.2 Secondary Objectives.....	12
Chapter 5 – Scope of the Project	13
5.1 In Scope (Version 2.0).....	13
5.2 Out of Scope (Deferred Beyond Phase-II)	13
5.3 Version 1.0 vs. Version 2.0 — At a Glance	13
Chapter 6 – Literature Review.....	14
6.1 Adaptive Learning Systems	14
6.2 Automated Question Generation	14
6.3 Identity and Session Management in Web Applications.....	14
6.4 Caching Strategies in Content-Generation Systems	14
6.5 Existing Platform Analysis (Updated).....	14
Chapter 7 – Proposed System (Version 2.0).....	15
7.1 System Concept.....	15
7.2 Key Differentiators (Updated)	15
7.3 System Boundaries	15

Chapter 8 – Requirement Analysis.....	17
8.1 Functional Requirements (Version 2.0).....	17
8.2 Non-Functional Requirements (Version 2.0).....	18
Chapter 9 – System Architecture.....	19
9.1 High-Level Architecture (Updated).....	19
9.2 Architectural Layers.....	19
9.2.1 Presentation Layer.....	19
9.2.2 API Layer.....	20
9.2.3 Data Layer.....	20
9.2.4 AI Inference Layer.....	20
Chapter 10 – Detailed Workflow.....	21
10.1 Authenticated Learning Phase Workflow.....	21
10.2 Adaptive Revision Workflow (Unchanged Logic, Authenticated Context).....	21
Chapter 11 – Database Design.....	22
11.1 Overview.....	22
11.2 Schema Evolution Summary.....	22
11.3 Updated Table Definitions.....	22
11.3.1 sessions Table.....	22
11.3.2 videos Table (Cache-Bearing).....	22
11.3.3 quizzes Table.....	22
11.3.4 quiz_attempts Table.....	23
11.4 Updated Entity Relationship Diagram.....	23
11.5 Indexes and Constraints.....	24
11.6 Why the Schema Was Not Further Normalised.....	24
Chapter 12 – Technology Stack Analysis.....	25
12.1 Updated Technology Stack.....	25
Chapter 13 – Frontend Implementation.....	26
13.1 Updated Page Structure.....	26
13.2 Authentication State Management (Replaced).....	27
13.3 Component Architecture (Extended).....	27
13.4 State Management Strategy.....	28
Chapter 14 – Backend Implementation and API Design.....	29
14.1 Updated API Route Architecture.....	29
14.2 /api/generate-notes Implementation (Updated).....	30
14.3 /api/generate-quiz Implementation (Updated).....	30
14.4 /api/generate-master-quiz Implementation (Unchanged, Now Authenticated).....	30
14.5 /api/delete-session and /api/delete-account (New).....	30
14.6 API Route Summary.....	30

Chapter 15 – AI Integration and Prompt Engineering	31
15.1 Model Selection (Unchanged).....	31
15.2 Prompt Engineering Strategy (Unchanged)	31
15.3 Impact of Caching on Token Consumption	31
15.4 Token Management (Unchanged)	31
Chapter 16 – Quiz Generation Engine	32
16.1 MCQ Data Schema (Unchanged)	32
16.2 Cache-Aware Quiz Lifecycle.....	32
16.3 Quiz Rendering (Unchanged, Navigation Improved).....	33
Chapter 17 – Performance Tracking and Adaptive Revision Engine	34
17.1 Weak Topic Detection Algorithm (Unchanged)	34
17.2 Revision Quiz Generation (Unchanged Logic, Now Ownership-Scoped)	34
17.3 Continuous Improvement Loop (Unchanged).....	34
Chapter 18 – Authentication Architecture	35
18.1 Migration Rationale.....	35
18.2 Authentication Flow	35
18.3 Server-Side Session Verification.....	36
18.4 User Profile Creation	36
18.5 Logout	36
18.6 Account Deletion	37
18.7 Removal of localStorage-Based Identity	37
18.8 Why This Architecture Is More Secure Than Phase-I.....	37
Chapter 19 – Intelligent Caching System	38
19.1 Motivation	38
19.2 Intelligent Notes Cache.....	38
19.3 Intelligent Quiz Cache.....	39
19.4 Why Master Revision Quizzes Remain Dynamic	39
19.5 Why Caching Is Keyed on yt_video_id Rather Than Local Database IDs	39
19.6 Cache Invalidation	39
19.7 Caching Architecture Summary	40
Chapter 20 – Session Ownership and Authorization	41
20.1 Ownership Model.....	41
20.2 Ownership Verification	41
20.3 Delete Session	42
20.4 Delete Account	43

Chapter 21 – Premium UI System.....	44
21.1 Design System Overview.....	44
21.2 Typography and Spacing.....	44
21.3 Dark Mode and Theme Provider.....	44
21.4 Sidebar Navigation.....	45
21.5 Dashboard Redesign.....	46
21.6 Loading Experience.....	47
21.7 Quiz, Revision, Landing, and Authentication Page Redesigns.....	47
21.8 Responsive and Accessibility Improvements.....	47
Chapter 22 – Analytics Architecture.....	49
22.1 Motivation for PostgreSQL RPCs.....	49
22.2 RPC-Based Aggregation.....	49
22.3 Why This Avoided Vercel Timeout Limits.....	49
22.4 Dashboard Statistics Display.....	50
Chapter 23 – Challenges Faced and Engineering Tradeoffs.....	51
23.1 Authentication Redirect Flickering.....	51
23.2 Supabase Migration and GoTrue Warnings.....	51
23.3 Duplicate Profiles and Database Race Conditions.....	51
23.4 Why SQL Constraints Initially Failed to Apply, and Why Old Duplicate Rows Blocked Migration.....	52
23.5 Repeated Transcript and Quiz Generation Prior to Caching.....	52
23.6 Dashboard Analytics Timeout and the PostgreSQL RPC Solution.....	52
23.7 Theme Provider Hydration and Tailwind Version Issues.....	52
23.8 Sidebar Responsiveness.....	53
23.9 Deployment Branch Management.....	53
23.10 Navigation Bugs and Loading Experience Redesign.....	53
23.11 Engineering Tradeoffs Analysis.....	54
Chapter 24 – Testing Strategy and Test Cases.....	55
24.1 Testing Approach (Updated).....	55
24.2 Updated Test Cases.....	55
Chapter 25 – Results and Discussion.....	56
25.1 Feature Completeness.....	56
25.2 Security.....	56
25.3 Performance and API Efficiency.....	56
25.4 Scalability.....	56
25.5 User Experience.....	56
25.6 Maintainability.....	56

Chapter 26 – Limitations	57
26.1 Known Limitations (Version 2.0)	57
Chapter 27 – Future Scope	58
27.1 Version 3.0 Development Priorities	58
Chapter 28 – Conclusion.....	59
References	60
Appendix.....	61
Appendix A: Project Statistics (Version 2.0).....	61
Appendix B: Environment Variables (Updated).....	61
Appendix C: Directory Structure (Updated).....	61
Appendix D: Figure Index	62
Appendix E: Table Index.....	62

Chapter 1 – Introduction

1.1 Project Overview

VibeLearn is an AI-powered personalised learning and adaptive revision platform that converts YouTube educational video content into structured notes, auto-generated quizzes, and a performance-driven revision cycle. The Phase-I MVP, completed in June 2026, validated this core learning loop with a deliberately lean monolithic architecture and a frictionless but insecure custom authentication mechanism. Version 2.0, the subject of this report, retains every feature of Phase-I while replacing its weakest architectural component — identity — with a production-grade authentication system, and uses that authenticated foundation to unlock a class of optimisation that was not previously possible: globally shared, cross-user caching of AI-generated content.

Where Phase-I treated every user's interaction with a given YouTube video as an isolated event — requiring its own transcript extraction and its own LLM inference call — Version 2.0 recognises that two different students studying the same lecture video should not each pay the full cost of note and quiz generation. This single architectural insight, made possible only once sessions and videos are reliably tied to verified user identities, is the central engineering theme of this phase.

1.2 Motivation for Phase-II

The Phase-I report explicitly flagged its authentication design as the single most significant limitation relative to a production-ready system, noting that any user who knew another user's username could access that user's data. This was an acceptable tradeoff for a four-week prototype sprint but an unacceptable one for a platform the author intended to use as the centrepiece of an internship portfolio and, eventually, as a genuinely usable personal study tool. Closing this gap was therefore the non-negotiable starting point of Phase-II.

A second motivation emerged organically once authentication work began: with a verified `user_id` available on the server for every request, it became straightforward to observe that the videos and quizzes table no longer needed to silently duplicate content across users. This observation reframed what was originally scoped as a pure security fix into a broader caching and database redesign effort.

1.3 Report Organisation

This report follows the same overall organisation as the Phase-I report — Background, Problem Statement, Objectives, Scope, Architecture, Requirements, Database, Frontend, Backend, AI Integration, Testing, Results, Limitations, Future Scope, and Conclusion — with every chapter updated to reflect the current state of the system. Five new chapters have been added where Phase-II introduced genuinely new subsystems: Authentication Architecture, Intelligent Caching System, Session Ownership and Authorization, Premium UI System, and Analytics Architecture.

Chapter 2 – Background and Motivation

2.1 The YouTube-as-Education Phenomenon

The background motivating VibeLearn's existence — the scale of educational content on YouTube and the well-documented gap between passive viewing and active learning — is unchanged from Phase-I and is not repeated in full here. What has changed is the author's own usage pattern: across the development of Phase-I and Phase-II, the author personally generated notes for several overlapping lecture videos while testing the platform, which made the wastefulness of per-user regeneration directly and repeatedly visible during day-to-day development.

2.2 The Forgetting Curve and Active Learning

The theoretical grounding in Ebbinghaus's forgetting curve and active-retrieval research, discussed in the Phase-I report, continues to motivate the quiz and revision subsystems unchanged in Version 2.0.

2.3 Limitations of Existing Solutions

The competitive gap identified in Phase-I — the absence of a platform combining automatic AI notes, AI MCQ generation, adaptive revision, and arbitrary YouTube compatibility — still holds. Version 2.0 widens this gap further by adding genuine multi-user efficiency: existing platforms that do offer AI note-taking (such as browser extensions built on top of ChatGPT) typically reprocess content per user with no shared caching layer, which VibeLearn now avoids.

2.4 Developer's Learning Goals for Phase-II

Beyond the educational problem, Phase-II served as a deliberate technical learning exercise in areas the author had explicitly deferred in Phase-I: production authentication flows (OAuth redirect handling, server-side session verification, GoTrue client behaviour), PostgreSQL constraint design under real data, and the use of database-side RPC functions to offload aggregation work that would otherwise risk serverless execution timeouts. Each of these is directly relevant to the author's target Software Development Engineer (SDE) and data-adjacent roles for the upcoming on-campus internship drives beginning in the 5th semester.

Chapter 3 – Problem Statement

3.1 Core Problem (Restated for Phase-II)

Phase-I solved the core pedagogical problem — absence of structured notes, lack of formative assessment, and no adaptive remediation — but in doing so introduced two new problems that became the explicit subject of Phase-II:

- Identity is not verifiable. Any actor who learns or guesses a username can impersonate that user and read or modify their sessions, notes, and quiz history.
- Computation and cost scale linearly, and wastefully, with users rather than with unique content. Every user who studies a popular YouTube video triggers a fresh transcript extraction and a fresh LLM generation call, even when another user generated functionally identical notes minutes earlier.

3.2 Problem Scope

Phase-II scope is bounded to: (a) replacing the identity layer end-to-end, including login, session persistence, logout, and account deletion; (b) introducing global, cross-user caching for AI-generated notes and per-video quizzes while preserving per-user dynamism for revision quizzes; and (c) redesigning the interface to a standard suitable for daily personal use and portfolio presentation. It explicitly excludes spaced-repetition scheduling, multi-language support, and collaborative sessions, all of which remain deferred to a genuine future phase (see Chapter 27).

3.3 Desired Outcome

A successful Phase-II would result in a platform where every piece of user data is provably owned by a verified identity, where the marginal cost of a second user studying an already-processed video approaches zero, and where the day-to-day experience of using the application no longer looks or feels like a four-week prototype.

Chapter 4 – Objectives

4.1 Primary Objectives (Phase-II)

- Migrate the entire authentication system from custom localStorage identity to Supabase Authentication with Google OAuth.
- Enforce server-side session verification and ownership checks on every protected API route.
- Design and implement a global, content-addressed caching layer for AI-generated notes and per-video quizzes, keyed on yt_video_id.
- Eliminate duplicate video and quiz generation across users while preserving full backward compatibility with the Phase-I learning workflow.
- Move dashboard analytics aggregation into PostgreSQL via RPC functions to avoid serverless timeout risk.

4.2 Secondary Objectives

- Redesign the interface around a coherent, theme-aware design system supporting dark mode.
- Introduce a collapsible sidebar navigation pattern in place of the Phase-I per-page header links.
- Implement safe, confirmable destructive actions: session deletion and full account deletion, both with cascading cleanup.
- Maintain the Phase-I application on the main branch in a deployable state throughout Phase-II development by isolating new work on a separate branch.
- Document the full project to the same professional software engineering report standard established in Phase-I.

Chapter 5 – Scope of the Project

5.1 In Scope (Version 2.0)

- Supabase Authentication with Google OAuth as the sole sign-in method.
- Server-side authenticated API routes using `supabase.auth.getUser()`.
- Global notes cache and global per-video quiz cache, both keyed on `yt_video_id`.
- Per-user dynamic master revision quizzes, generated fresh on every request.
- Session ownership enforcement and cascading delete for sessions and accounts.
- Dark mode, sidebar navigation, and a redesigned visual system across all pages.
- PostgreSQL RPC-based dashboard analytics.

5.2 Out of Scope (Deferred Beyond Phase-II)

- Spaced repetition scheduling (SM-2 / FSRs).
- Multi-language transcript and prompt support.
- Collaborative or shared sessions across multiple users.
- Offline mode and Progressive Web App (PWA) packaging.
- Mobile-native application.
- Payment integration or subscription management.

5.3 Version 1.0 vs. Version 2.0 — At a Glance

Table 1 summarises, at a high level, the principal differences between the Phase-I MVP and the Version 2.0 system described in this report. Each item is expanded in its corresponding chapter.

Dimension	Version 1.0 (Phase-I)	Version 2.0 (Phase-II)
Identity	Custom username, localStorage	Supabase Auth, Google OAuth, server-verified
Content generation	Per-user, always fresh	Global cache for notes and video quizzes
Dashboard analytics	Not present / minimal	PostgreSQL RPC-aggregated stats
UI theming	Ad hoc Tailwind classes	Semantic tokens, dark mode via next-themes
Navigation	Per-page header links	Persistent collapsible sidebar
Destructive actions	Not implemented	Confirmable session and account deletion

Table 1: Version 1.0 to Version 2.0 Comparison Overview

Chapter 6 – Literature Review

6.1 Adaptive Learning Systems

The discussion of Intelligent Tutoring Systems and Bayesian Knowledge Tracing presented in the Phase-I report remains the relevant theoretical backdrop for VibeLearn's weak-topic-driven revision engine, which is functionally unchanged in Version 2.0 aside from the analytics layer described in Chapter 22.

6.2 Automated Question Generation

The literature on LLM-based MCQ generation cited in Phase-I (Elkins et al., 2023) continues to inform prompt design in Version 2.0; no change to the underlying generation methodology was made in this phase.

6.3 Identity and Session Management in Web Applications

Phase-II required engaging, for the first time in this project, with the literature and documentation surrounding modern authentication patterns: the OAuth 2.0 Authorization Code flow, JSON Web Token (JWT) based session representation, and the GoTrue identity service that underpins Supabase Authentication. Industry guidance from the OWASP (Open Worldwide Application Security Project) Authentication Cheat Sheet was consulted informally during design discussions, particularly its emphasis on never trusting client-supplied identity claims and always re-verifying session tokens server-side — a principle directly reflected in VibeLearn's exclusive use of `supabase.auth.getUser()` on the server rather than trusting any client-passed `user_id`.

6.4 Caching Strategies in Content-Generation Systems

The decision to cache AI-generated outputs by content identity rather than by requesting user draws on general content-addressable caching principles common in CDN (Content Delivery Network) and build-system design, where a cache key derived from the input (here, `yt_video_id`) rather than the requester allows safe sharing of computed results across unrelated callers. This is conceptually distinct from typical per-user response caching and was the central design decision behind Chapter 19 of this report.

6.5 Existing Platform Analysis (Updated)

Platform	Auto Notes	MCQ Gen	Adaptive	Cross-User Cache	Free
Coursera	No	Partial	Partial	N/A	Partial
Khan Academy	No	Yes	Yes	N/A	Yes
Notion AI	Manual	No	No	No	Partial
Quizlet	No	Partial	No	No	Partial
VibeLearn v2.0	Yes (AI)	Yes (AI)	Yes	Yes	Yes

Table 2: Comparison of Existing Learning Platforms (Updated for Version 2.0)

Chapter 7 – Proposed System (Version 2.0)

7.1 System Concept

Version 2.0 retains the two-phase learning model from Phase-I — a Learning Phase and an Adaptive Revision Phase — but inserts an authentication and cache-resolution step ahead of both. Every workflow now begins by establishing a verified user identity, and the Learning Phase now begins by checking whether the requested video has already been processed by any user before deciding whether to invoke the transcript extraction and LLM pipeline at all.

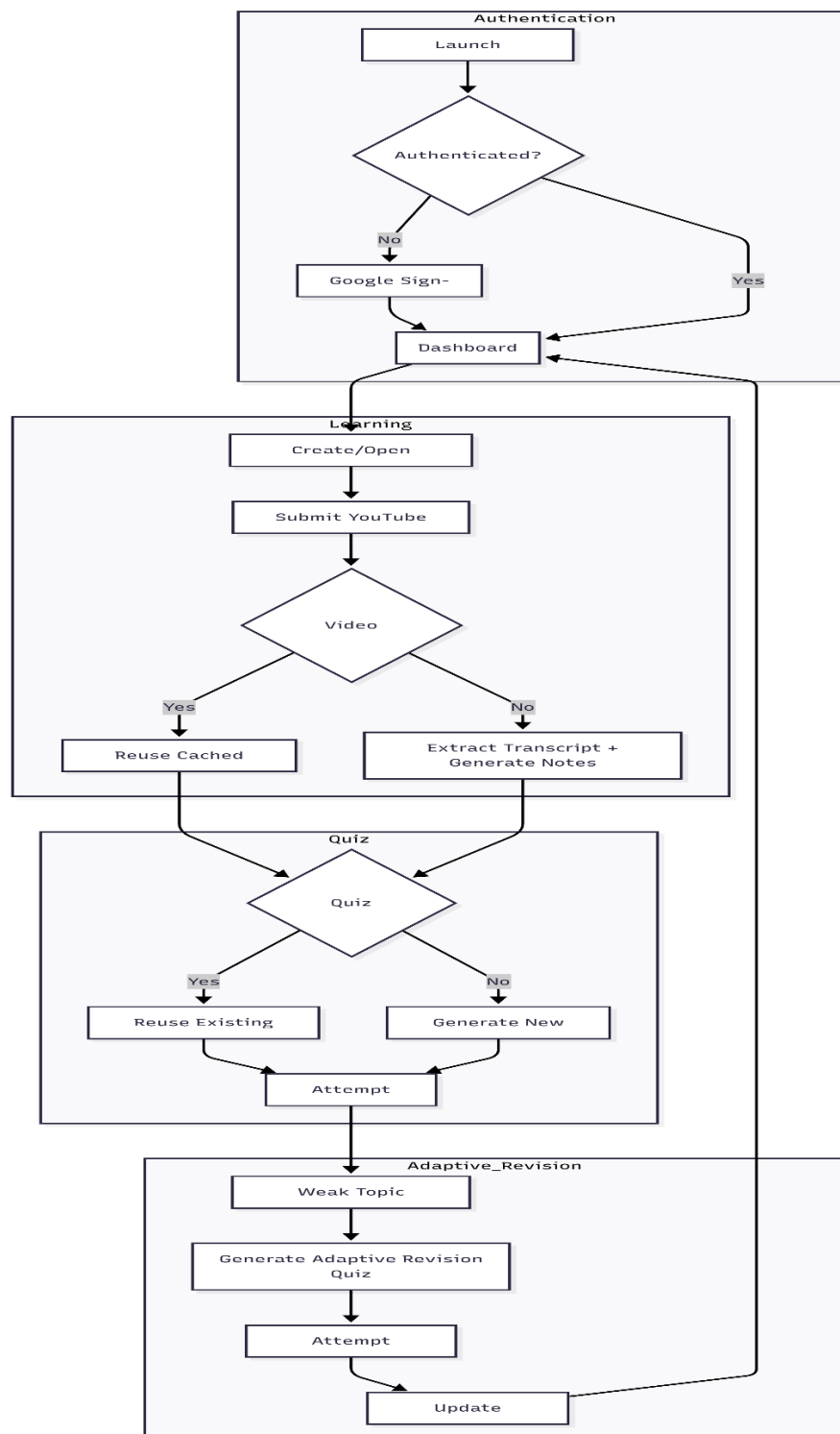
7.2 Key Differentiators (Updated)

- Full automation, unchanged from Phase-I: no manual note-taking or quiz authoring required.
- Content grounding, unchanged: generated notes and questions are grounded in actual video content.
- Verified ownership: every session, video reference, and quiz attempt is bound to a Supabase-authenticated identity.
- Global efficiency: AI generation cost is paid once per unique video across the entire user base, not once per user.
- Session-level adaptive revision, generated fresh per request rather than cached, preserving personalisation where it matters most.

7.3 System Boundaries

The system boundary is extended relative to Phase-I to include the Supabase Authentication service (GoTrue) and the Google OAuth identity provider as external dependencies. The Groq API, Supabase PostgreSQL database, and YouTube caption servers remain external dependencies as before.

Figure 7.1
VibeLearn Learning and Revision Lifecycle (Version 2.0)



Chapter 8 – Requirement Analysis

8.1 Functional Requirements (Version 2.0)

Table 3 lists the functional requirements newly introduced or materially changed in Phase-II. Requirements FR01–FR14 carried over from the Phase-I report remain in force unchanged and are not repeated here for brevity.

ID	Module	Requirement	Priority
FR15	Auth	System shall authenticate users exclusively via Supabase Authentication with Google OAuth	High
FR16	Auth	System shall verify the session server-side via <code>supabase.auth.getUser()</code> on every protected route	High
FR17	Auth	System shall support logout, invalidating the client session	High
FR18	Auth	System shall support full account deletion with cascading removal of all owned data	High
FR19	Session Mgmt	System shall verify session ownership before returning or mutating any session-scoped data	High
FR20	Notes Cache	System shall check for an existing notes record for a given <code>yt_video_id</code> before generating new notes	High
FR21	Notes Cache	System shall reuse cached notes across unrelated users and sessions when present	High
FR22	Quiz Cache	System shall check for an existing <code>video_quiz</code> record for a given <code>yt_video_id</code> before generating a new quiz	High
FR23	Quiz Cache	System shall always regenerate <code>revision_master</code> quizzes dynamically per request	High
FR24	Database	System shall enforce a uniqueness constraint preventing duplicate video rows for the same <code>yt_video_id</code>	High
FR25	Session Mgmt	System shall support session deletion with cascading removal of dependent quiz and attempt records	High
FR26	Dashboard	System shall display aggregated session statistics computed via PostgreSQL RPC	Medium
FR27	UI	System shall support a persistent, collapsible sidebar for primary navigation	Medium
FR28	UI	System shall support a user-toggleable dark mode persisted across sessions	Medium
FR29	UI	System shall support a Notes Only viewing mode and a Video Only viewing mode that are mutually exclusive	Low

Table 3: Functional Requirements (Phase-II additions; FR01–FR14 from Phase-I remain in force)

8.2 Non-Functional Requirements (Version 2.0)

ID	Category	Requirement	Priority
NFR09	Security	No API key, service-role credential, or unverified user_id shall ever be trusted from client input	High
NFR10	Security	Session tokens shall be managed exclusively by the Supabase Auth client; no custom token logic shall be written	High
NFR11	Performance	Cached note or quiz retrieval shall return in under 1 second, materially faster than the 15-second NFR01 generation bound	High
NFR12	Reliability	Dashboard aggregation queries shall complete within Vercel's serverless execution time limit under realistic data volumes	High
NFR13	Maintainability	All new authentication and caching logic shall be isolated in dedicated modules rather than interleaved into existing route handlers	Medium
NFR14	Usability	Theme preference (light/dark) shall persist across page reloads without a flash of incorrect theme	Medium

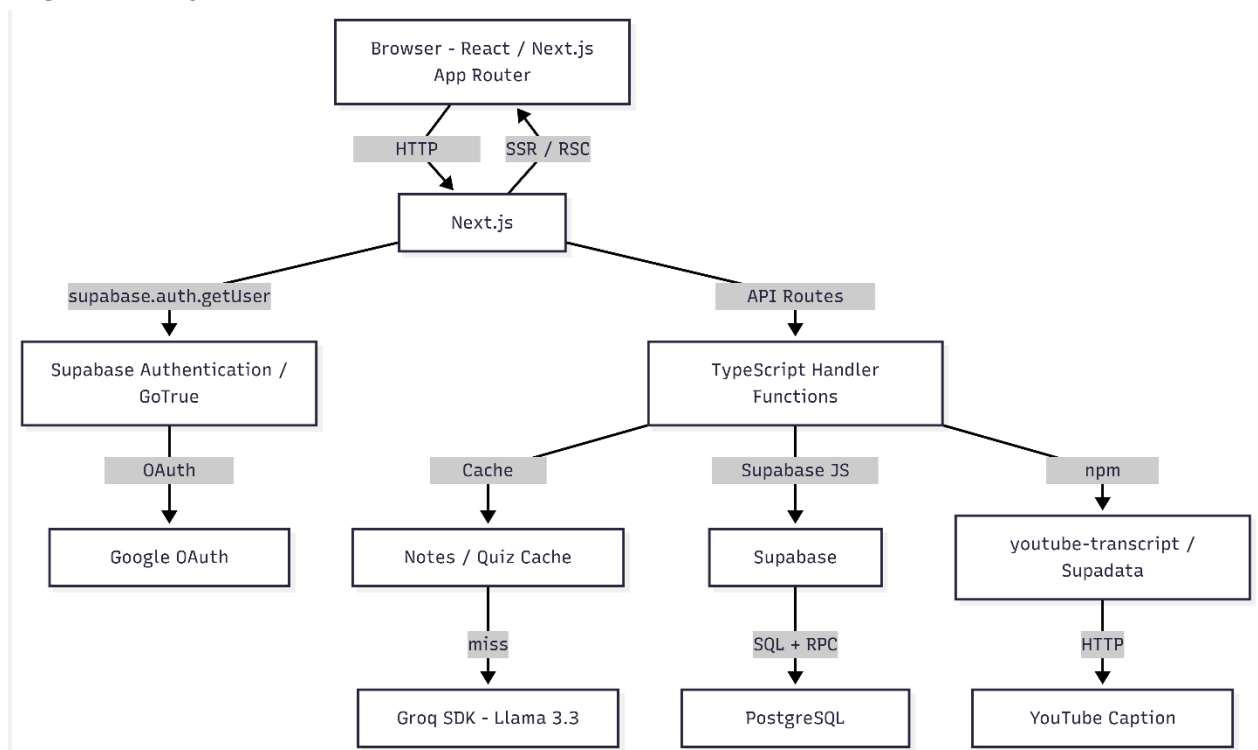
Table 4: Non-Functional Requirements (Phase-II additions; NFR01–NFR08 from Phase-I remain in force)

Chapter 9 – System Architecture

9.1 High-Level Architecture (Updated)

VibeLearn remains a monolithic Next.js application; this architectural choice, justified in Phase-I on the grounds of deployment simplicity and rapid iteration, was deliberately not revisited in Phase-II. Introducing microservices, a message queue, or a separate authentication server was considered and explicitly rejected: the added operational complexity was judged disproportionate to the benefit for a single-developer, Vercel-deployed application. What changed is the internal request path: every API route handler now begins with a session-verification step against Supabase Authentication before any business logic executes, and the note/quiz generation handlers now begin with a cache-lookup step before any external AI call is made.

Figure 9.1
High-Level System Architecture, Version 2.0



9.2 Architectural Layers

9.2.1 Presentation Layer

Unchanged in fundamentals from Phase-I — Next.js App Router pages composed of Server and Client Components, styled with Tailwind CSS and shadcn/ui — but substantially expanded with a sidebar shell component, a ThemeProvider wrapping the application root, and redesigned page-level layouts described in Chapter 21.

9.2.2 API Layer

Every Route Handler in app/api now follows a consistent four-step pattern: authenticate, authorize (ownership check where applicable), resolve from cache where applicable, and execute. This is a deliberate generalisation of the Phase-I pattern of input validation, external service invocation, persistence, and response.

9.2.3 Data Layer

The data layer continues to be Supabase-managed PostgreSQL, now additionally hosting Supabase Authentication's internal auth.users table, two PL/pgSQL RPC functions for dashboard aggregation, and a uniqueness constraint on videos.yt_video_id supporting the caching strategy described in Chapter 19.

9.2.4 AI Inference Layer

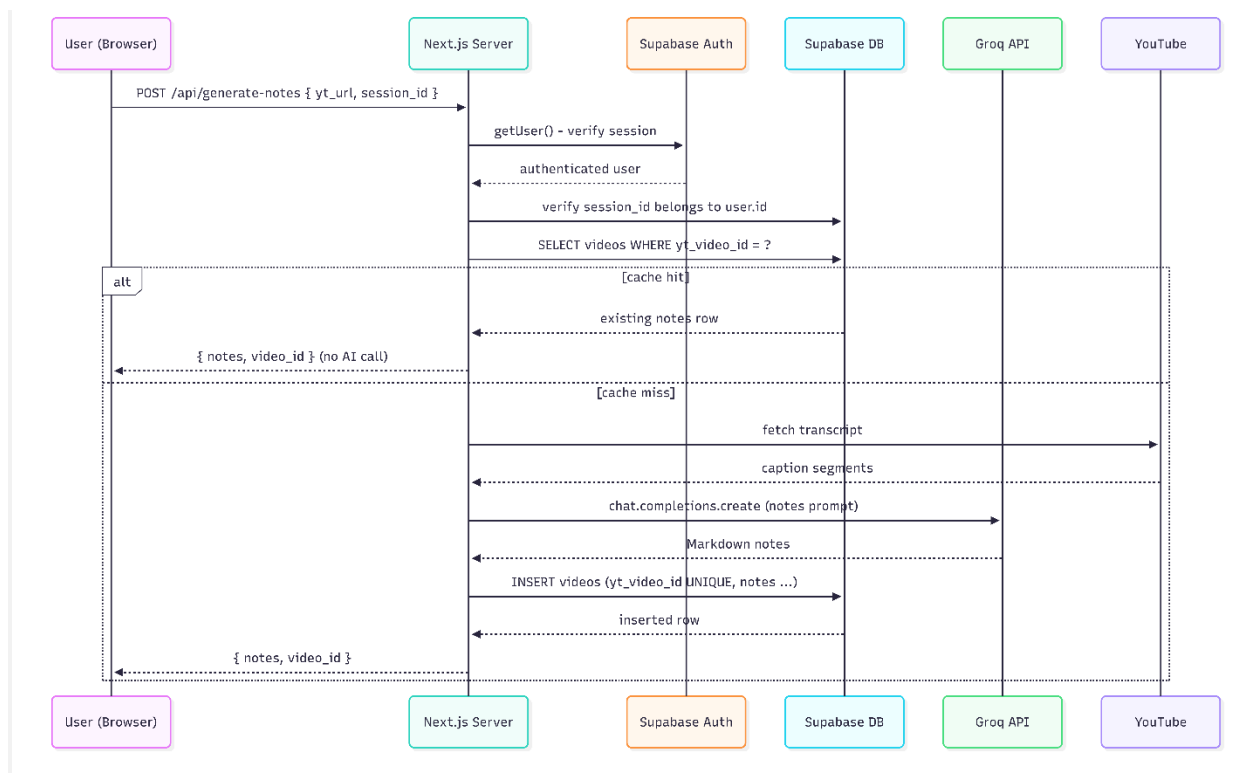
Unchanged: the Groq API serving Llama 3.3 70B Versatile remains the sole inference provider. What changed is the frequency with which this layer is invoked, which is now gated by the caching layer described in Chapter 19.

Chapter 10 – Detailed Workflow

10.1 Authenticated Learning Phase Workflow

Figure 10.1

Authenticated, Cache-Aware Learning Phase Sequence



10.2 Adaptive Revision Workflow (Unchanged Logic, Authenticated Context)

The weak-topic aggregation and revision quiz generation logic described in the Phase-I report is unchanged in Version 2.0. The only addition is that every database read in this workflow is now scoped to session and quiz rows whose ownership has already been verified earlier in the request, and the resulting revision_master quiz is, by design, never served from cache, since its value lies precisely in being tailored to the requesting user's accumulated weak topics.

Chapter 11 – Database Design

11.1 Overview

The database architecture of VibeLearn Version 2.0 was redesigned to improve security, scalability, and overall system performance. Compared to Version 1.0, the updated schema integrates Supabase Authentication, introduces intelligent caching for AI-generated content, and strengthens ownership validation through improved relationships and constraints. The database is implemented using PostgreSQL on Supabase and consists of six primary application tables, with authentication managed separately by Supabase Auth.

11.2 Schema Evolution Summary

Table	Phase-I	Version 2.0 Change
users	Application user records	Legacy application user table retained for backward compatibility; authentication is now managed by Supabase Auth.
profiles	Not present	Added to store user profile information synchronized after authentication
sessions	Basic session management	Enhanced with secure ownership validation
videos	Per-session storage	Global caching using unique YouTube Video IDs
quizzes	Generated for every request	Video quizzes globally cached using YouTube Video IDs
quiz_attempts	Stored quiz scores	Extended for adaptive revision and weak-topic tracking

Table 5: Database Schema Evolution, Phase-I to Version 2.0

11.3 Updated Table Definitions

11.3.1 sessions Table

```
CREATE TABLE sessions (  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  title TEXT NOT NULL,  status TEXT NOT NULL DEFAULT 'active',  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

11.3.2 videos Table (Cache-Bearing)

```
CREATE TABLE videos (  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  session_id UUID REFERENCES sessions(id) ON DELETE CASCADE,  yt_video_id TEXT NOT NULL UNIQUE,  yt_url TEXT NOT NULL,  title TEXT,  thumbnail_url TEXT,  notes TEXT,  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

11.3.3 quizzes Table

```
CREATE TABLE quizzes (  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  session_id UUID REFERENCES sessions(id) ON DELETE CASCADE,  video_id UUID REFERENCES videos(id) ON DELETE SET NULL,  yt_video_id TEXT,  quiz_type TEXT NOT NULL,  questions JSONB NOT NULL,  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

11.3.4 quiz_attempts Table

```
CREATE TABLE quiz_attempts (  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  quiz_id
UUID NOT NULL REFERENCES quizzes(id) ON DELETE CASCADE,  user_id UUID NOT NULL REFERENCES
auth.users(id) ON DELETE CASCADE,  score INTEGER NOT NULL,  total_questions INTEGER NOT
NULL,  weak_topics JSONB,  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW() );
```

11.3.5 profiles Table

```
CREATE TABLE profiles (  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
username TEXT UNIQUE,, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW());
```

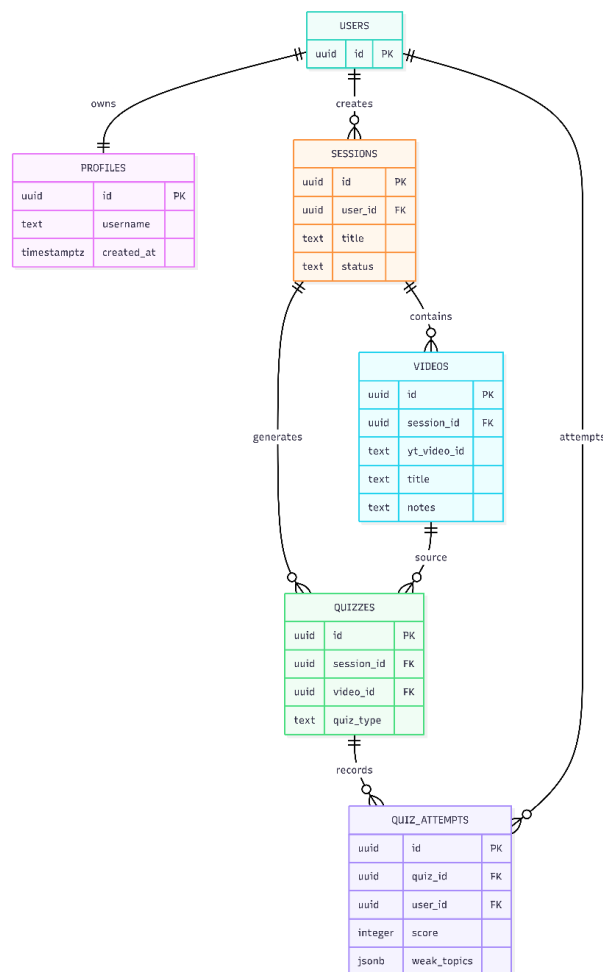
11.3.6 users Table

```
CREATE TABLE users (  id UUID PRIMARY KEY DEFAULT gen_random_uuid(), username TEXT NOT
NULL,  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW());
```

11.4 Updated Entity Relationship Diagram

Figure 11.1

Entity Relationship Diagram, Version 2.0



11.5 Indexes and Constraints

Version 2.0 introduces several database-level constraints to improve data integrity and maintain consistency across the application. The profiles table uses a shared primary key with `auth.users(id)`, enforcing a strict one-to-one relationship between an authenticated user and the corresponding application profile. UUID-based primary keys uniquely identify each record, while foreign key relationships enforce referential integrity between users, sessions, videos, quizzes, and quiz attempts. A unique constraint on the YouTube Video ID prevents duplicate processing of identical educational videos, enabling intelligent caching of AI-generated notes and quizzes and reducing redundant API calls.

11.6 Why the Schema Was Not Further Normalised

Although additional normalization through intermediary mapping tables could further improve scalability, the current schema was intentionally kept simpler to balance implementation complexity and application requirements. The existing design provides efficient data retrieval, supports intelligent caching, and remains maintainable while satisfying the functional requirements of Version 2.0. More advanced normalization strategies may be considered in future versions as the application scales.

Chapter 12 – Technology Stack Analysis

12.1 Updated Technology Stack

Technology	Category	Rationale	Status
Next.js App Router	Full-Stack Framework	Unified frontend/backend, SSR, API routes	Unchanged from Phase-I
TypeScript	Language	Type safety across larger, more complex codebase	Unchanged
Tailwind CSS	Styling	Utility-first styling, now driving a semantic token system	Extended (Ch. 21)
shadcn/ui	Component Library	Accessible primitives, now themed for dark mode	Extended
Lucide React	Icons	Consistent icon set for sidebar and UI redesign	New in Phase-II
React Markdown	Rendering	Notes rendering, unchanged	Unchanged
next-themes	Theming	Dark mode persistence without flash-of-incorrect-theme	New in Phase-II
Supabase	BaaS / Database	PostgreSQL backbone, now also hosting Authentication	Extended
Supabase Authentication	Identity	Google OAuth, server-side session verification	New in Phase-II
Groq API	LLM Inference	High-throughput inference, call volume now reduced by caching	Unchanged provider
Llama 3.3 70B Versatile	Language Model	Unchanged model choice	Unchanged
Supadata	Transcript Extraction	Supplementary transcript retrieval used alongside youtube-transcript	New in Phase-II
Vercel	Deployment	Unchanged hosting target	Unchanged

Table 6: Technology Stack Summary, Version 2.0

No technology was removed in this phase. The introduction of Supadata as a secondary transcript extraction path was a direct response to the transcript retrieval failures documented in the Phase-I Challenges chapter, providing a fallback when the youtube-transcript package's reliance on undocumented YouTube endpoints results in a failed or rate-limited request.

Chapter 13 – Frontend Implementation

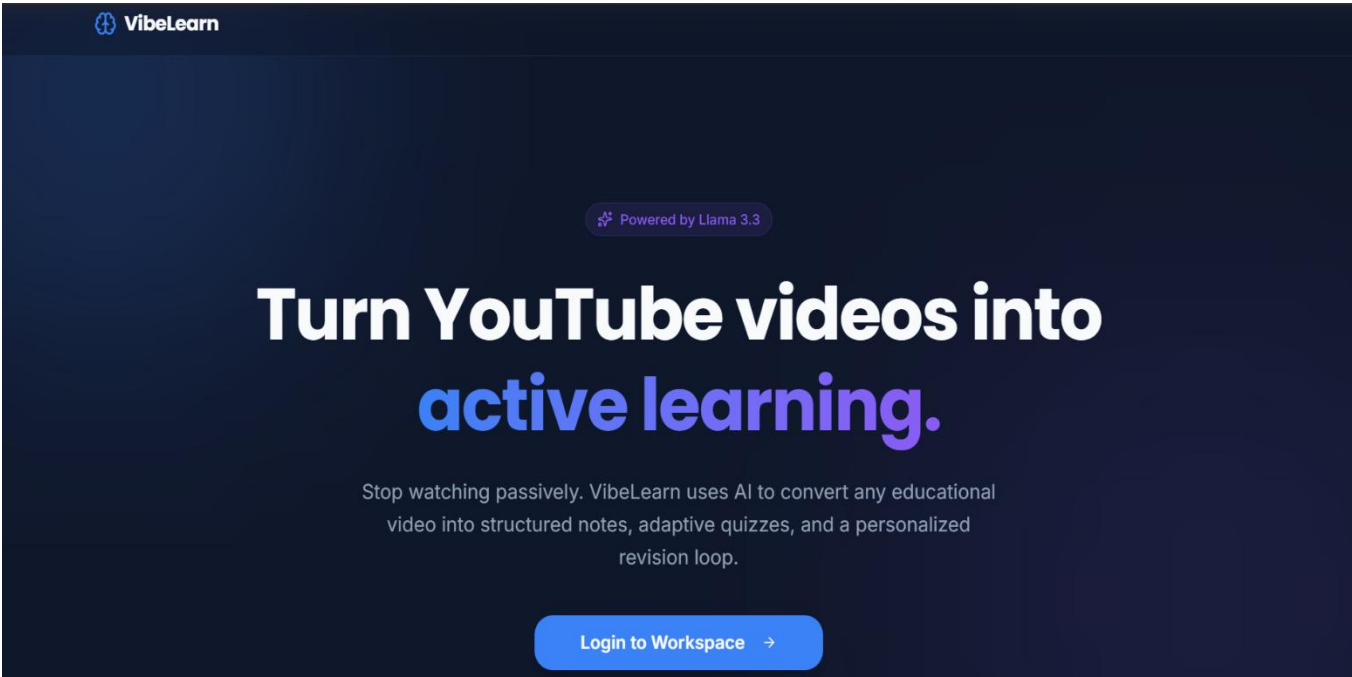
13.1 Updated Page Structure

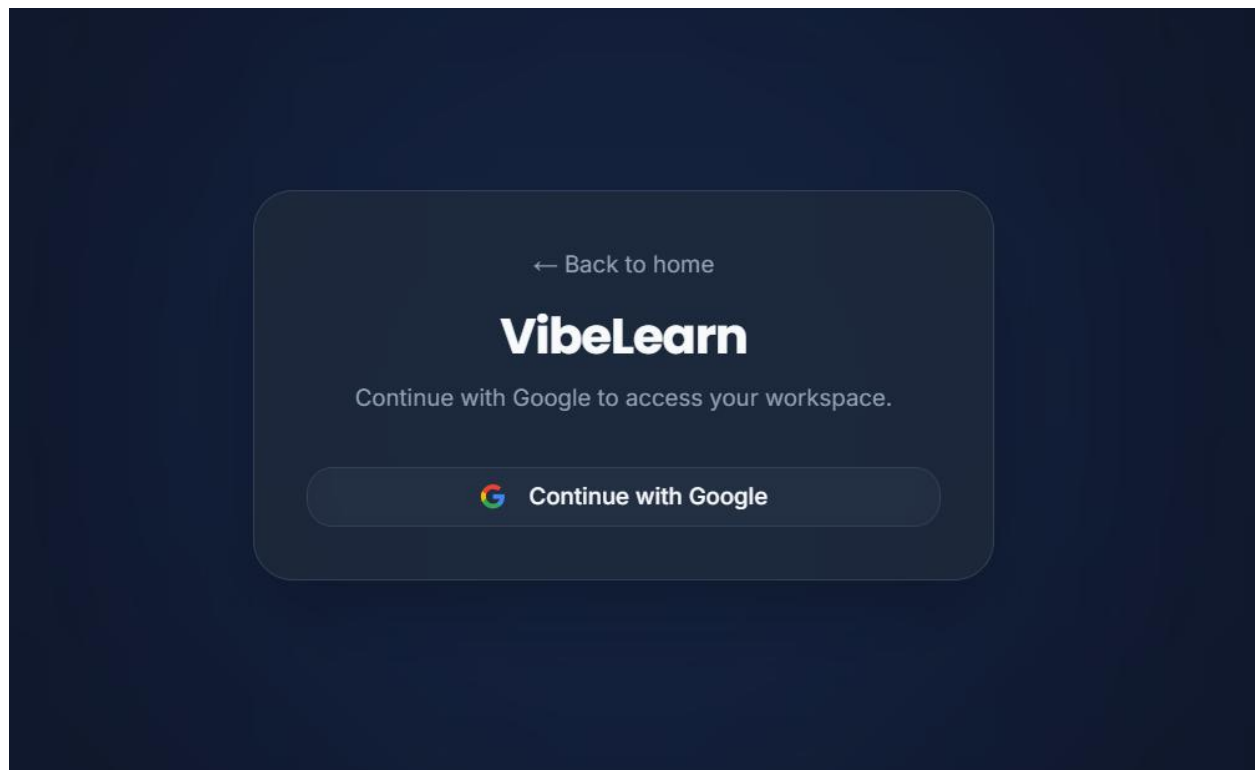
The five core routes from Phase-I (home, dashboard, session workspace, quiz, revision) are retained, with the home route repurposed as the Supabase Authentication entry point rather than a username form, and a new /auth/callback route added to handle the OAuth redirect.

Route	Purpose in Version 2.0
/	Landing and sign-in page; presents a Sign in with Google button rather than a username form
/auth/callback	Route Handler that exchanges the OAuth code for a session and redirects to the dashboard
/dashboard	Redesigned hub with sidebar navigation, analytics cards, and session list
/session/[id]	Session workspace with Notes Only / Video Only mode toggle
/quiz/[id]	Redesigned quiz interface with improved exit/finish navigation
/revision/[id]	Redesigned revision interface, unchanged core logic

Table 7: Application Routes, Version 2.0

Figure 13.1
Landing Page





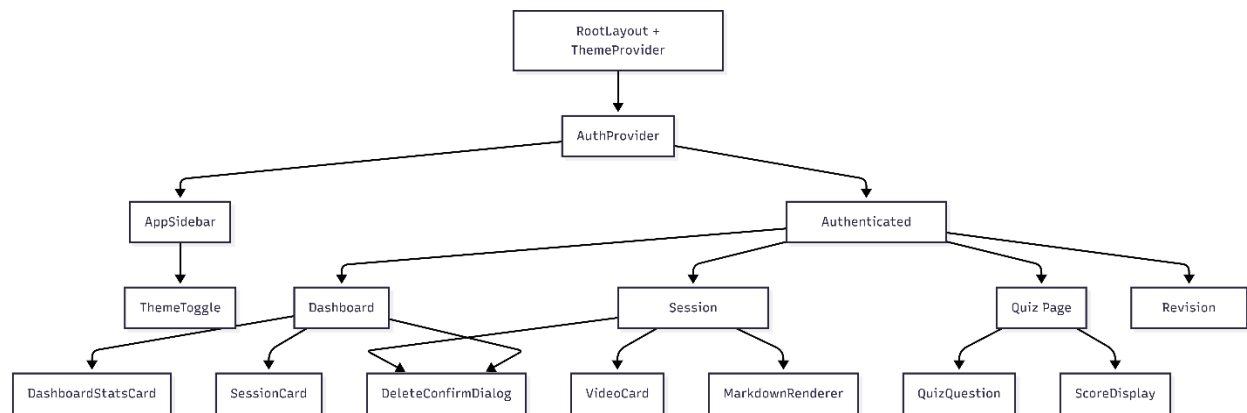
13.2 Authentication State Management (Replaced)

The Phase-I AuthContextProvider, which read a UUID and username directly from localStorage on mount, has been entirely removed. In its place, an AuthProvider built on the Supabase JS client subscribes to onAuthStateChanged and exposes the current session and user object via a useAuth hook with an identical call signature to the Phase-I version, which kept the migration's blast radius on downstream components small. Internally, however, the source of truth is now the Supabase session object, refreshed automatically by the Supabase client's token-refresh logic, rather than a static value read once from browser storage.

13.3 Component Architecture (Extended)

The Phase-I component set (VideoCard, QuizQuestion, ScoreDisplay, SessionCard, MarkdownRenderer) is retained. New components introduced in Phase-II include: AppSidebar (collapsible navigation shell), ThemeToggle (light/dark switch consuming next-themes), LoadingScreen (full-screen loading state with rotating motivational copy, described in Chapter 21), DeleteConfirmDialog (a reusable confirmation dialog used for both session deletion and account deletion), and DashboardStatsCard (renders a single aggregated metric returned by the analytics RPC).

Figure 13.2
Frontend Component Hierarchy, Version 2.0



13.4 State Management Strategy

Unchanged in philosophy from Phase-I: local component state via `useState/useReducer` for transient UI state, and no global state management library. The theme value is the one new piece of cross-cutting state, and it is deliberately handled by `next-themes` rather than by a hand-rolled context, since `next-themes` specifically solves the flash-of-incorrect-theme problem that a naïve React Context implementation would not.

Chapter 14 – Backend Implementation and API Design

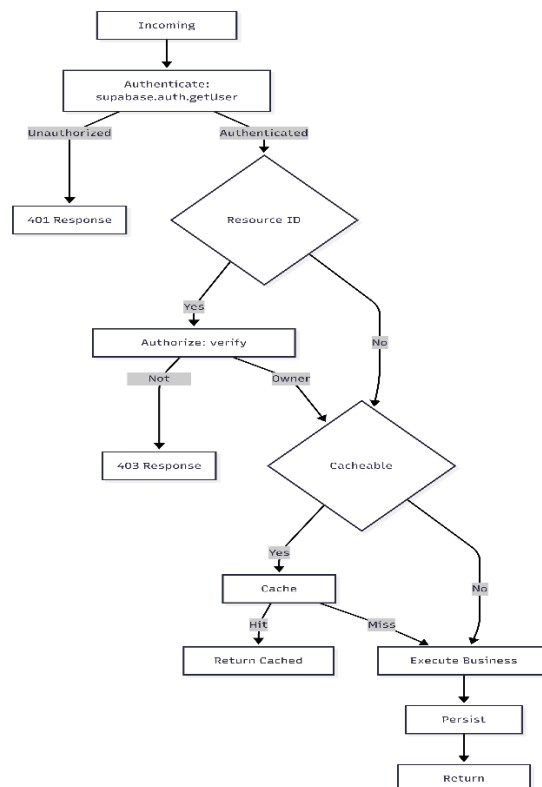
14.1 Updated API Route Architecture

Every Route Handler now begins with an identical authentication preamble before any route-specific logic executes:

```
const supabase = createServerClient(); const { data: { user }, error } = await
supabase.auth.getUser(); if (error || !user) { return NextResponse.json({ error:
'Unauthorized' }, { status: 401 }); }
```

This pattern was deliberately centralised into a shared helper after being copy-pasted across the first three migrated routes, both to reduce duplication and to ensure that a future route could not accidentally be left unauthenticated by omission. This is a direct, practical application of the Don't Repeat Yourself principle to a security-critical code path: centralising the check means a future bug fix or hardening change to the authentication preamble propagates automatically to every route that uses it, rather than needing to be manually re-applied across an ever-growing number of copy-pasted blocks.

Figure 14.1
Protected API Route Request Flow



14.2 /api/generate-notes Implementation (Updated)

The route now performs a cache check immediately after authentication and ownership verification: it queries videos for an existing row matching the extracted `yt_video_id`. On a cache hit, the existing notes are returned immediately and the `session_id` parameter is used only to associate the cached video with the requesting user's session for display purposes, without touching the Groq API or the transcript extraction pipeline at all. On a cache miss, the Phase-I pipeline (transcript retrieval, AI inference, metadata enrichment, persistence) proceeds unchanged, with the new `yt_video_id` UNIQUE constraint as a safety net against a race condition discussed in Chapter 23.

14.3 /api/generate-quiz Implementation (Updated)

Analogous cache-check logic was added: before invoking the LLM, the handler queries quizzes for an existing row with `quiz_type = 'video_quiz'` and a matching `yt_video_id`. If found, that quiz is reused, and only a fresh association between the quiz and the requesting user's session is persisted as needed. If not found, generation proceeds exactly as in Phase-I.

14.4 /api/generate-master-quiz Implementation (Unchanged, Now Authenticated)

This route is deliberately excluded from the caching strategy. It continues to execute the full weak-topic aggregation and generation pipeline on every call, since its entire value proposition is personalisation to the requesting user's specific quiz history; caching it would defeat its purpose. The only Phase-II change to this route is the addition of the authentication preamble and a `user_id`-scoped query for `quiz_attempts` in place of the unscoped query used in Phase-I.

14.5 /api/delete-session and /api/delete-account (New)

Two new routes were added to support the destructive-action workflows described in Chapter 20. Both verify ownership before acting: `delete-session` confirms the `session.user_id` matches the authenticated user before issuing a cascading delete on the sessions row, which PostgreSQL propagates to dependent videos, quizzes, and `quiz_attempts` rows via the ON DELETE CASCADE constraints defined in Chapter 11. `delete-account` performs the same ownership-based cascading deletion at the user level by deleting the corresponding `auth.users` row through the Supabase Admin API, which in turn cascades through every table referencing `auth.users(id)`.

14.6 API Route Summary

Route	Method	Auth Required	Cache-Aware
/api/generate-notes	POST	Yes	Yes (notes cache)
/api/generate-quiz	POST	Yes	Yes (video_quiz cache)
/api/generate-master-quiz	POST	Yes	No — always dynamic
/api/delete-session	POST	Yes	N/A
/api/delete-account	POST	Yes	N/A
/auth/callback	GET	N/A (establishes session)	N/A

Table 8: API Route Summary, Version 2.0

Chapter 15 – AI Integration and Prompt Engineering

15.1 Model Selection (Unchanged)

Llama 3.3 70B Versatile via Groq remains the sole inference provider in Version 2.0; no alternative model was evaluated or adopted in this phase.

15.2 Prompt Engineering Strategy (Unchanged)

The role-specification, explicit-format, schema-definition, quantity-specification, and quality-instruction principles documented in the Phase-I report continue to govern all three AI-driven prompts unchanged.

15.3 Impact of Caching on Token Consumption

The most significant AI-integration change in Phase-II is not to the prompts themselves but to the frequency with which they are sent. Because notes and per-video quizzes are now generated once per unique video rather than once per user-video pair, repeated study of popular videos — which the author observed informally during testing, particularly with short introductory programming tutorials — no longer incurs repeated Groq token cost. This is discussed qualitatively, to the extent informal observation allows, in Chapter 25; no formal token-usage benchmarking infrastructure was built in this phase, so the discussion is intentionally framed in terms of directional engineering reasoning rather than fabricated figures.

15.4 Token Management (Unchanged)

The 128,000-token context window of Llama 3.3 70B Versatile continues to comfortably accommodate the transcript lengths encountered in practice. The chunk-and-summarise strategy flagged as future work in Phase-I remains unimplemented and is retained in the Future Scope chapter of this report.

Chapter 16 – Quiz Generation Engine

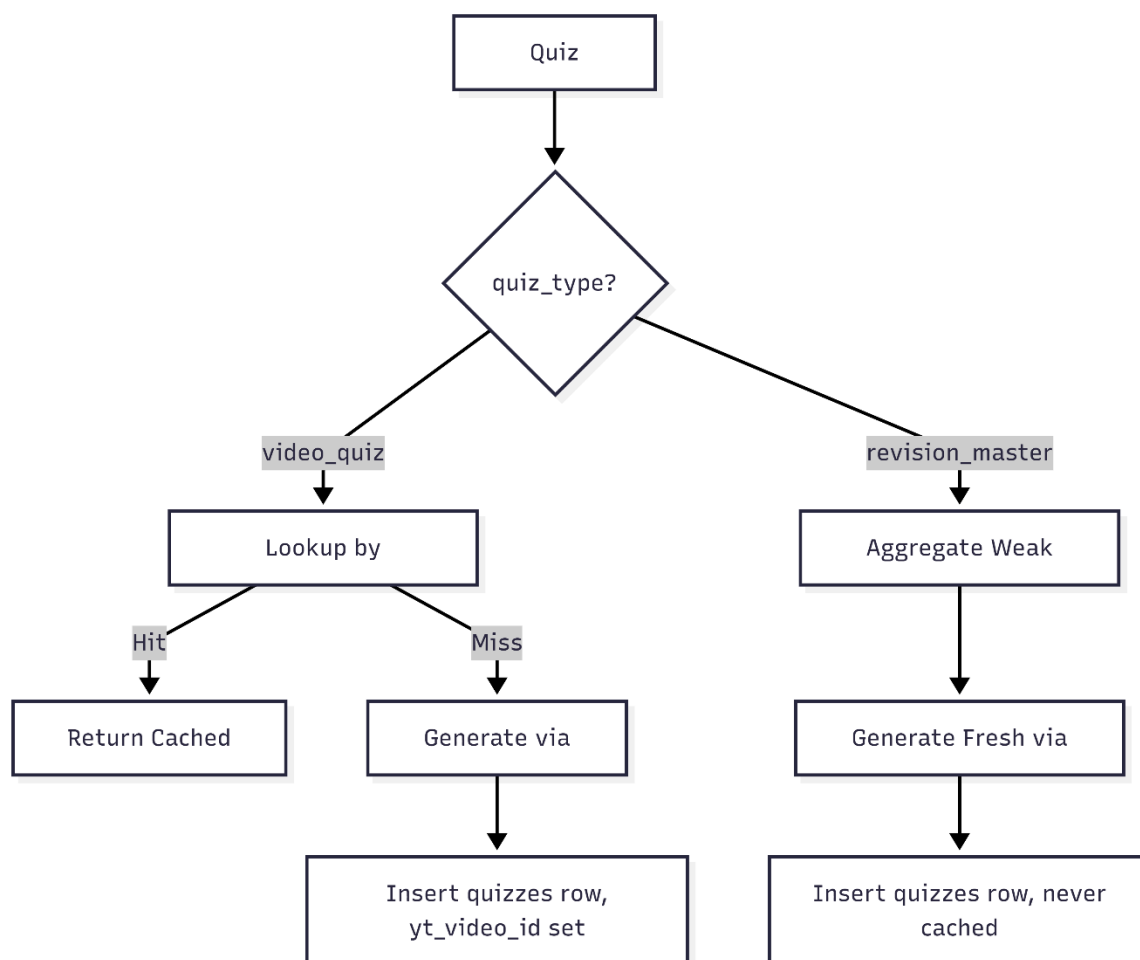
16.1 MCQ Data Schema (Unchanged)

The question object schema — question, options, correctIndex, explanation, topic — is unchanged from Phase-I and continues to underpin both video_quiz and revision_master quiz types.

16.2 Cache-Aware Quiz Lifecycle

video_quiz rows now have a dual identity: they are addressed both by their own UUID (for direct retrieval when a user revisits a session) and by the yt_video_id of their parent video (for cache resolution when a different user studies the same video for the first time). revision_master rows retain only the UUID addressing scheme from Phase-I, since, as established in Chapter 14, they are intentionally never cached or reused.

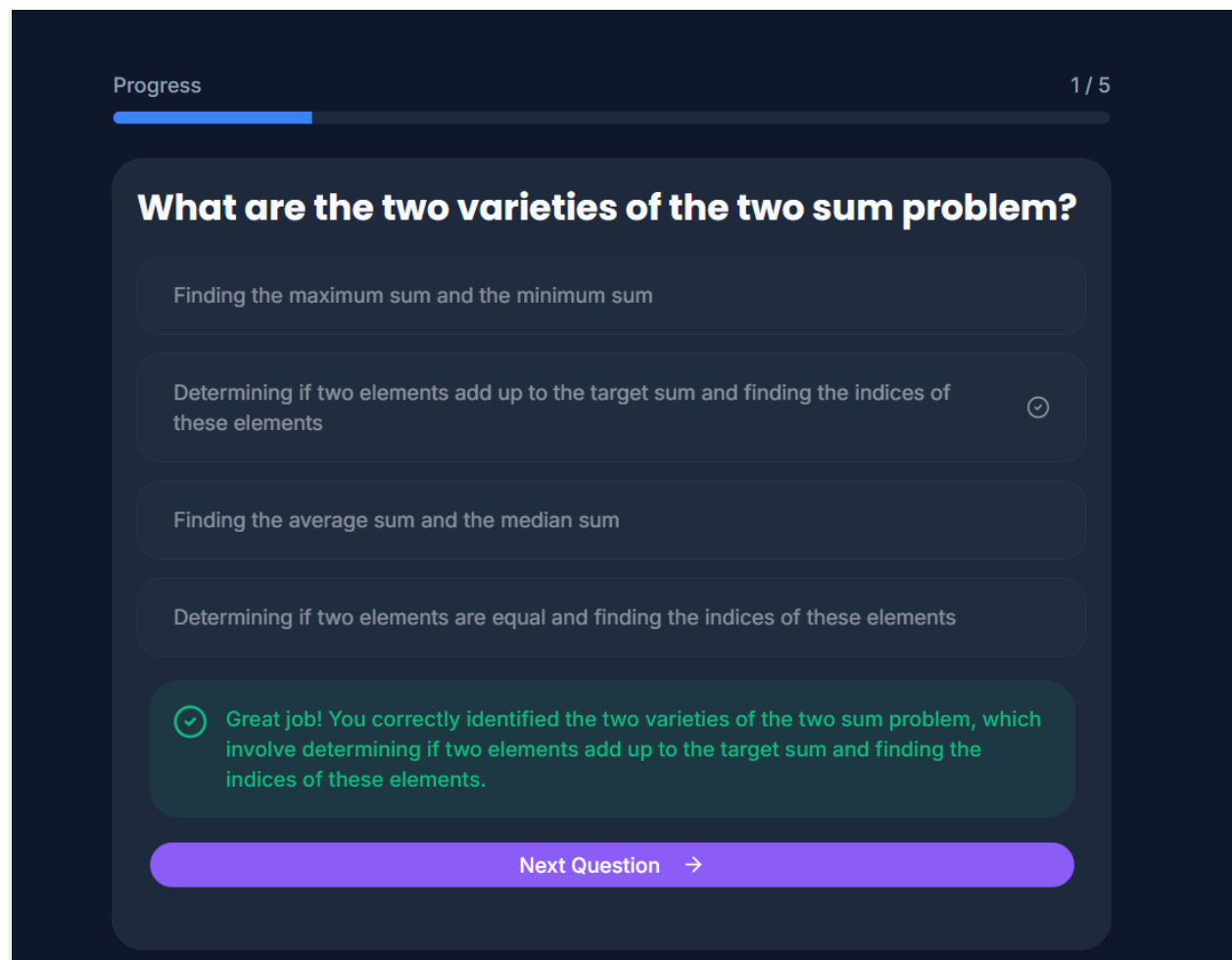
Figure 16.1
Cache-Aware Quiz Lifecycle



16.3 Quiz Rendering (Unchanged, Navigation Improved)

The instant-evaluation rendering behaviour described in Phase-I is unchanged. The surrounding navigation was improved in Phase-II: exiting a quiz now returns the user to the active study session they originated from rather than to the dashboard, and finishing a quiz correctly routes back to the session workspace, both addressed as part of the navigation bugs discussed in Chapter 23.

Figure 16.2
Quiz Interface



Chapter 17 – Performance Tracking and Adaptive Revision Engine

17.1 Weak Topic Detection Algorithm (Unchanged)

The Set-based deduplication algorithm for weak topic aggregation, documented in full in the Phase-I report, is unchanged in Version 2.0.

17.2 Revision Quiz Generation (Unchanged Logic, Now Ownership-Scoped)

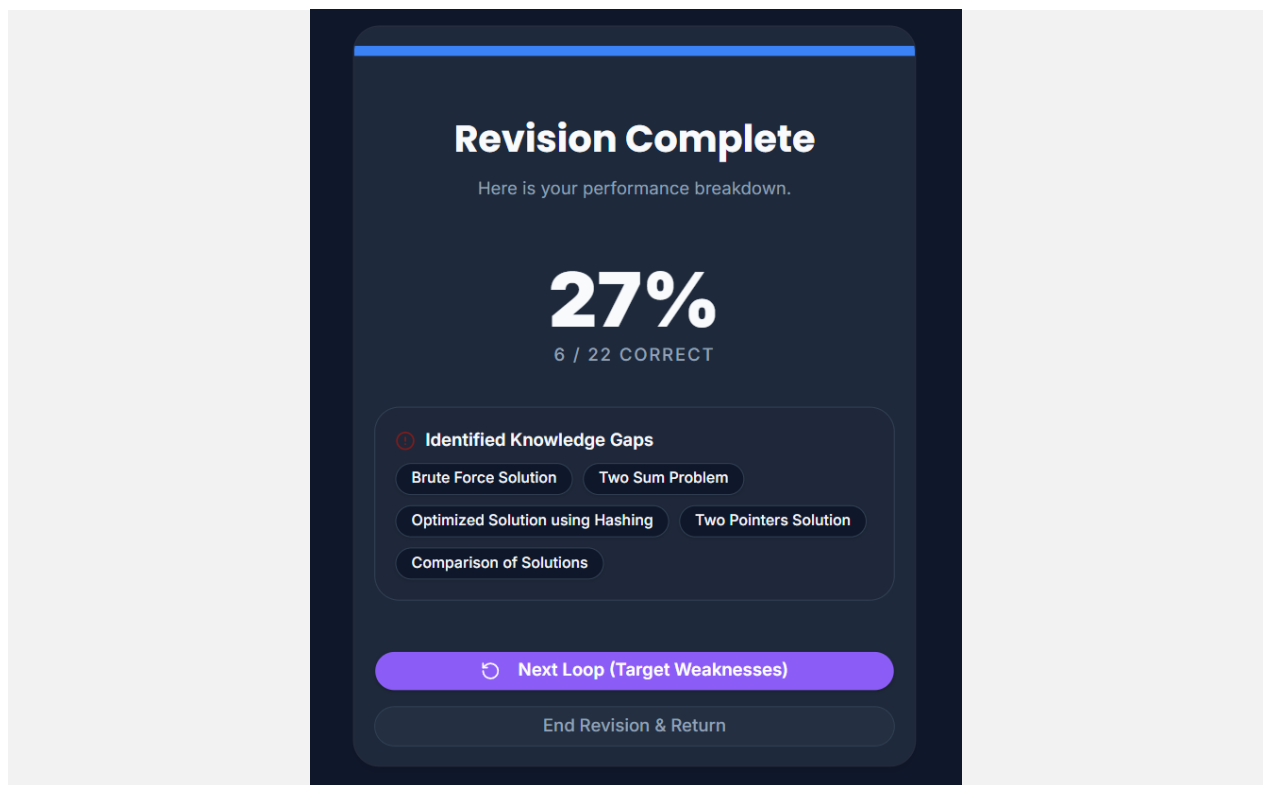
The revision quiz prompt construction — combining deduplicated weak topics with concatenated session notes — is unchanged. The query that gathers prior quiz_attempts for a session is now additionally scoped to the authenticated user_id, which, combined with the session ownership check performed earlier in the request, ensures a user can never have their revision quiz influenced by another user's quiz history even if session and quiz identifiers were somehow guessed.

17.3 Continuous Improvement Loop (Unchanged)

The loop described in Phase-I, where each revision attempt feeds into the weak-topic pool for the next, continues to operate identically in Version 2.0.

Figure 17.1

Adaptive Revision Dashboard



Chapter 18 – Authentication Architecture

18.1 Migration Rationale

This chapter documents the single largest engineering effort of Phase-II: the complete removal of the Phase-I custom authentication mechanism and its replacement with Supabase Authentication. The Phase-I report's own Future Authentication Roadmap section identified Supabase Auth with OAuth as the leading candidate for this work, and that proposal was carried through into implementation largely as originally envisaged, with Google OAuth chosen in preference to GitHub OAuth for accessibility to a broader, non-technical set of potential users beyond the author's immediate developer peer group.

Supabase Authentication was preferred over a hand-rolled JWT implementation for a concrete engineering reason: session token issuance, refresh, and revocation are notoriously easy to get subtly wrong, and any defect in custom token logic would reintroduce exactly the class of identity vulnerability this phase set out to eliminate. Delegating this responsibility to a managed identity service (GoTrue) removes an entire category of bugs from the author's own codebase, at the cost of an external dependency — a tradeoff judged clearly favourable for a security-critical subsystem.

18.2 Authentication Flow

Figure 18.1

Google OAuth Authentication Callback Flow

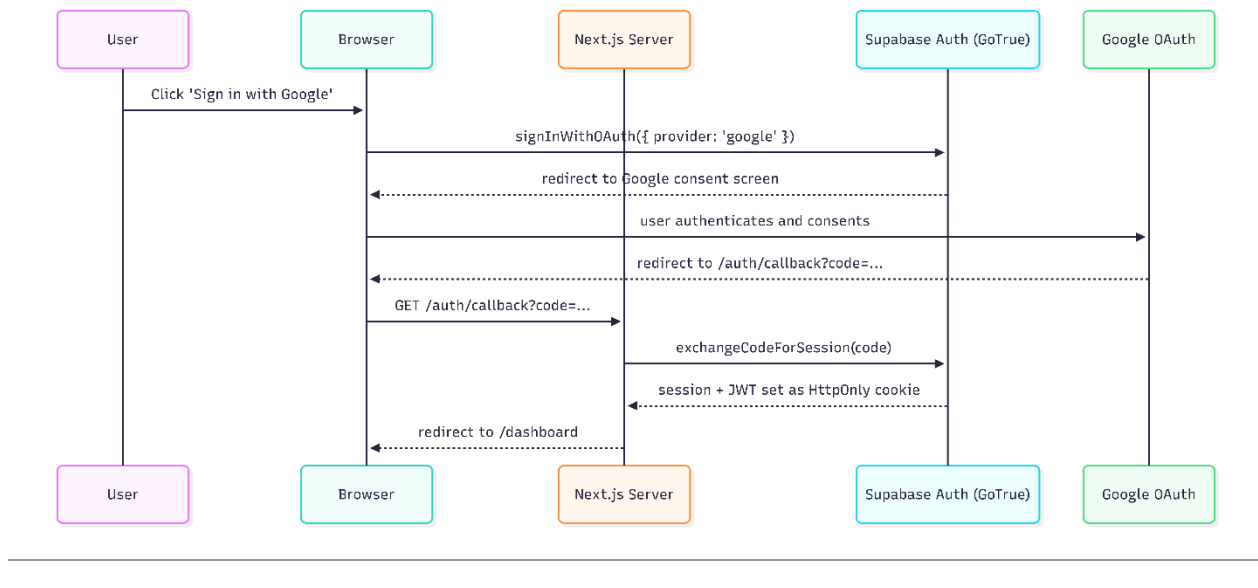
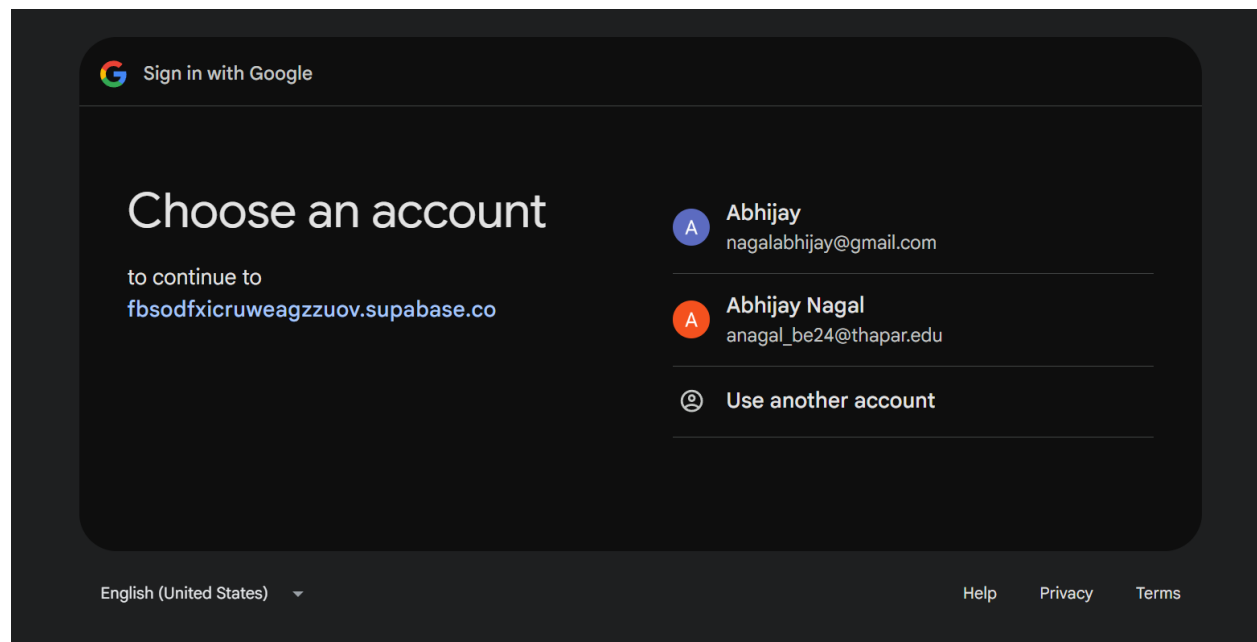


Figure 18.2
Google OAuth Consent Screen



18.3 Server-Side Session Verification

Every protected API route and every server-rendered protected page calls `supabase.auth.getUser()` rather than reading any client-supplied identifier. This single design rule is the structural fix for the Phase-I identity-spoofing vulnerability: because `getUser()` re-validates the session token against the Supabase Authentication service on the server, there is no value the client can fabricate or replay that would cause the server to believe it is a different user than the one actually holding a valid session.

18.4 User Profile Creation

During the authentication process, a PostgreSQL trigger automatically creates a corresponding record in the profiles table using data received from Supabase Authentication. The table stores application-specific information such as the username while authentication remains managed by `auth.users`.

18.5 Logout

Logout calls `supabase.auth.signOut()`, which clears the session cookie and revokes the refresh token client-side, after which the AuthProvider's `onAuthStateChanged` listener updates application state and the user is redirected to the landing page.

18.6 Account Deletion

Account deletion is exposed as a clearly labelled, confirmation-gated action (see Chapter 20) that calls the dedicated `/api/delete-account` route described in Chapter 14, which uses the Supabase Admin client to delete the user's `auth.users` row, triggering the cascading deletes configured throughout the schema in Chapter 11.

18.7 Removal of localStorage-Based Identity

Every read and write of identity data to `localStorage` present in the Phase-I codebase was removed in this phase. The only remaining authentication state is managed internally by the Supabase Auth client, which is responsible for securely handling session lifecycle, token refresh, and expiry. Unlike Phase-I, the application itself no longer reads, writes, or trusts user identity stored in `localStorage`, eliminating the custom client-managed identity mechanism.

18.8 Why This Architecture Is More Secure Than Phase-I

Property	Phase-I	Version 2.0
Identity verification	None; username typed by anyone	Cryptographically signed session token, re-verified server-side
Session storage	<code>localStorage</code> , script-accessible	Managed by the Supabase Auth client; application code no longer manages authentication state directly.
Impersonation risk	Trivial; type any existing username	Requires compromising the OAuth provider account or a valid session token
Server trust model	Trusted client-supplied <code>user_id</code>	Server independently re-validates identity on every request

Table 9: Security Comparison, Phase-I vs. Version 2.0 Authentication

Chapter 19 – Intelligent Caching System

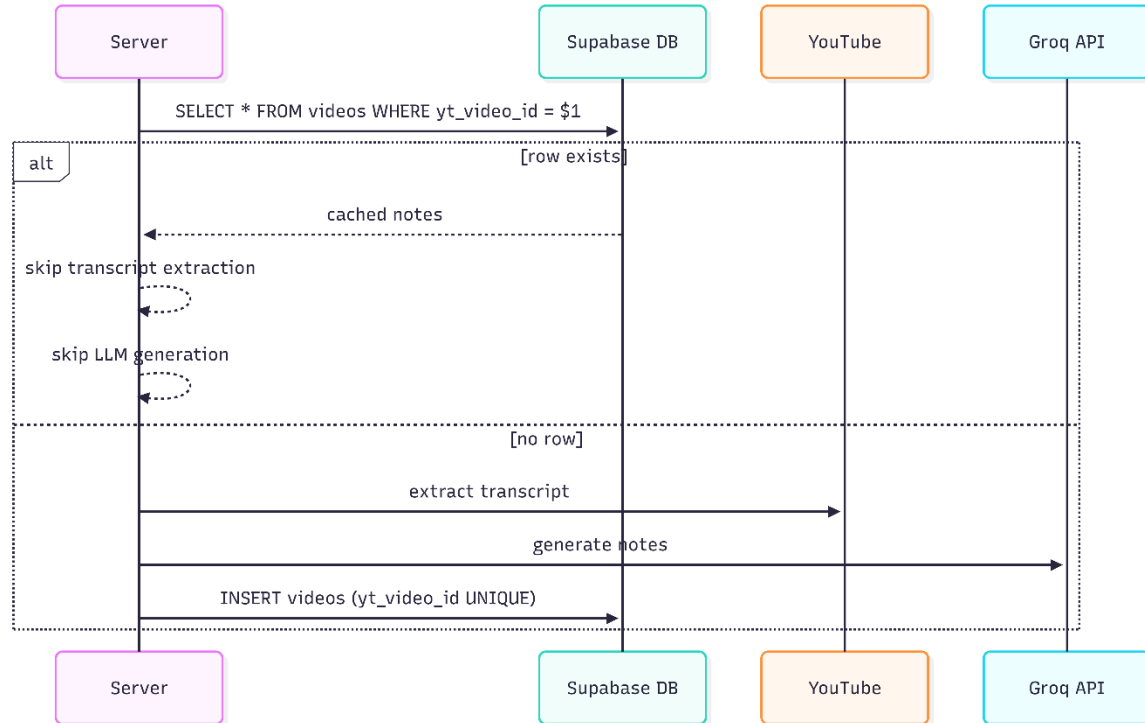
19.1 Motivation

In Phase-I, every user who added a given YouTube video to a session triggered an independent transcript extraction and an independent LLM call to generate notes, and a further independent LLM call to generate that video's quiz. Two users studying the same popular tutorial would, under that design, each pay the full latency and token cost, and would each receive notes and quizzes that were functionally equivalent but textually distinct, with no benefit to either user from the other's prior work. This was an explicit, accepted limitation in Phase-I, justified there as out of scope for a four-week prototype. It became the primary engineering target of Phase-II once authentication made it possible to safely distinguish a user's session from the underlying shared video content.

19.2 Intelligent Notes Cache

The notes cache is implemented as the videos table itself, with `yt_video_id` elevated from an ordinary column to a `UNIQUE` constraint. The `/api/generate-notes` handler, after authenticating the requester and verifying session ownership, queries for an existing videos row matching the extracted `yt_video_id` before doing anything else.

Figure 19.1
Intelligent Notes Cache Resolution



On a cache hit, transcript extraction is skipped entirely, LLM generation is skipped entirely, the Groq API is not called, and the response returns directly from the database read. The engineering consequence is threefold: API call volume to both the YouTube caption endpoints and the Groq API is reduced for any video studied by more than one user; Groq token usage decreases proportionally to the degree of overlap in studied content across the user base; and Vercel serverless execution time for the route drops from several seconds (dominated by transcript fetch and LLM latency) to well under a second (a single indexed database read), which in turn reduces both cost and the user-perceived latency of adding an already-known video to a new session.

19.3 Intelligent Quiz Cache

The quiz cache applies the identical pattern to quizzes rows where `quiz_type = 'video_quiz'`, resolved by the denormalised `yt_video_id` column discussed in Chapter 11.3.3. As with the notes cache, a second user requesting a quiz for an already-quizzed video receives the existing quiz directly, with no LLM call.

19.4 Why Master Revision Quizzes Remain Dynamic

`revision_master` quizzes are deliberately excluded from this caching strategy. Their entire purpose, as established in Phase-I, is to be conditioned on a specific user's accumulated weak topics across their own quiz attempts; two different users with different weak topics must, by the definition of the feature, receive different revision quizzes even when studying from the same underlying session notes. Caching this quiz type would silently break the adaptive revision feature, so the implementation explicitly bypasses the cache-lookup step for any request where `quiz_type` is `revision_master`.

19.5 Why Caching Is Keyed on `yt_video_id` Rather Than Local Database IDs

An early implementation considered keying the cache on the local `videos.id` (a UUID generated per insert), but this was rejected once it became clear that the entire point of the cache is to let unrelated sessions, with unrelated local video rows, converge on the same underlying content. `yt_video_id` is the only identifier in the system that is stable across users and sessions and genuinely identifies the same real-world piece of content, which makes it the only correct cache key for this purpose.

19.6 Cache Invalidation

No active cache invalidation mechanism was implemented in Phase-II. Cached notes and quizzes are treated as permanent once generated, on the reasoning that the underlying YouTube video content for a given video ID does not change after publication in the vast majority of cases, and that the cost of incorrect staleness (a rare edited re-upload under the same ID, which YouTube itself does not generally support for the same video ID) was judged lower than the complexity of building an invalidation system in this phase. This is flagged explicitly as an accepted limitation in Chapter 26.

19.7 Caching Architecture Summary

Content Type	Cache Key	Shared Across Users?	Invalidation
Notes (videos.notes)	yt_video_id	Yes	None (permanent)
Per-video quiz (video_quiz)	yt_video_id	Yes	None (permanent)
Revision quiz (revision_master)	N/A — never cached	No — always personalised	N/A

Table 10: Caching Architecture Summary, Version 2.0

Chapter 20 – Session Ownership and Authorization

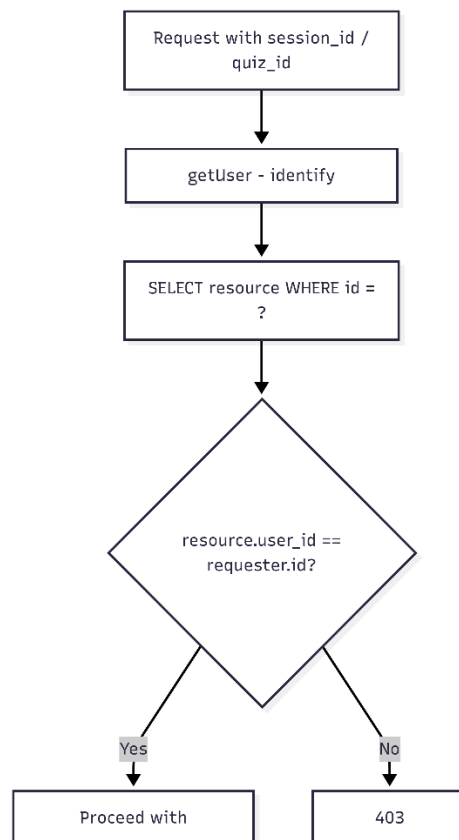
20.1 Ownership Model

Every sessions row carries a `user_id` foreign key to `auth.users(id)`, established at creation time from the authenticated request context rather than from any client-supplied value. Ownership of a session implicitly extends to every video, quiz, and quiz_attempt reachable through that session via the foreign-key chain described in Chapter 11, with the exception of the globally cached videos and video_quiz quizzes content itself, which, as discussed in Chapter 19, is shared rather than owned.

20.2 Ownership Verification

Every route handler that accepts a `session_id`, `quiz_id`, or similar resource identifier performs an explicit ownership check — a `SELECT` confirming the resource's `user_id` (or its owning session's `user_id`) matches the authenticated `user.id` — before performing any read or write beyond that check itself. A request for a resource owned by a different user receives a 403 Forbidden response rather than the requested data or a generic 404, a deliberate choice to make authorization failures unambiguous during development and testing.

Figure 20.1
Session Ownership Verification



20.3 Delete Session

Session deletion is exposed via a destructive button in the session workspace, gated behind a confirmation dialog (DeleteConfirmDialog) that requires an explicit second click before the `/api/delete-session` route is called. The route re-verifies ownership before issuing the delete, and relies on the ON DELETE CASCADE constraints on `videos.session_id`, `quizzes.session_id`, and `quiz_attempts.quiz_id` to clean up dependent rows automatically, with the explicit exception that a cached videos row whose `yt_video_id` is still referenced by another user's session is not deleted, since the cascade is scoped through the session relationship rather than through the globally shared video content.

Figure 20.2
Delete Session Flow

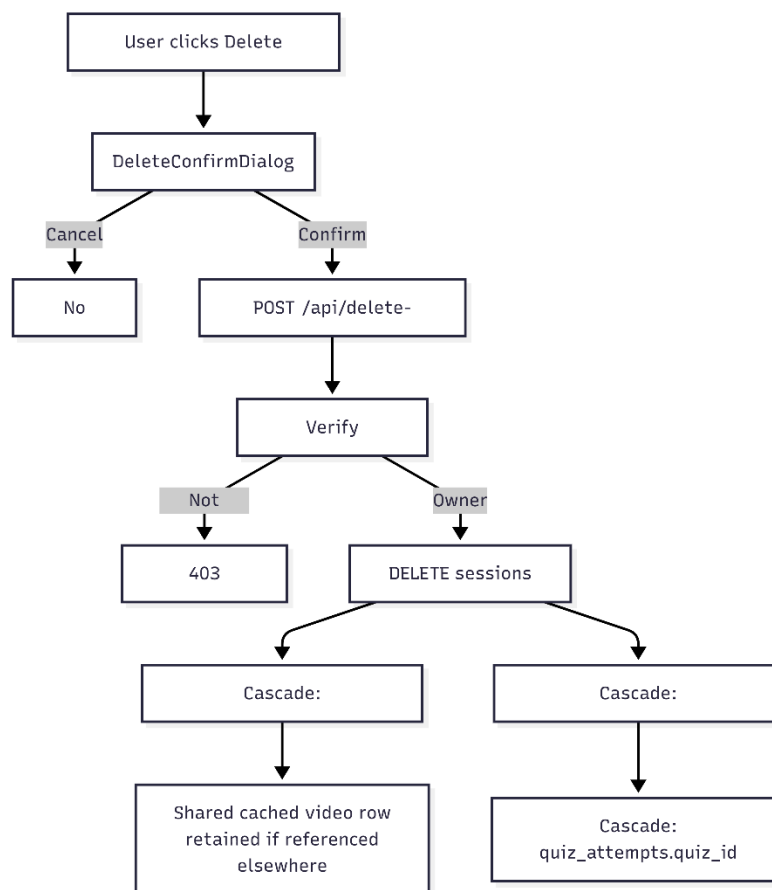
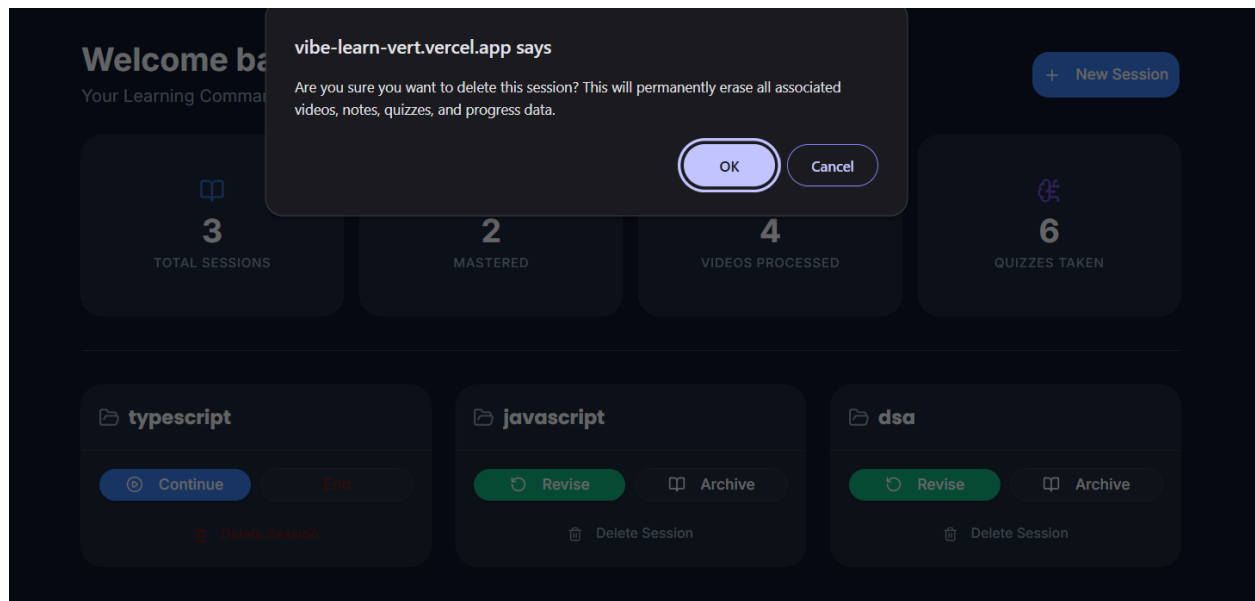


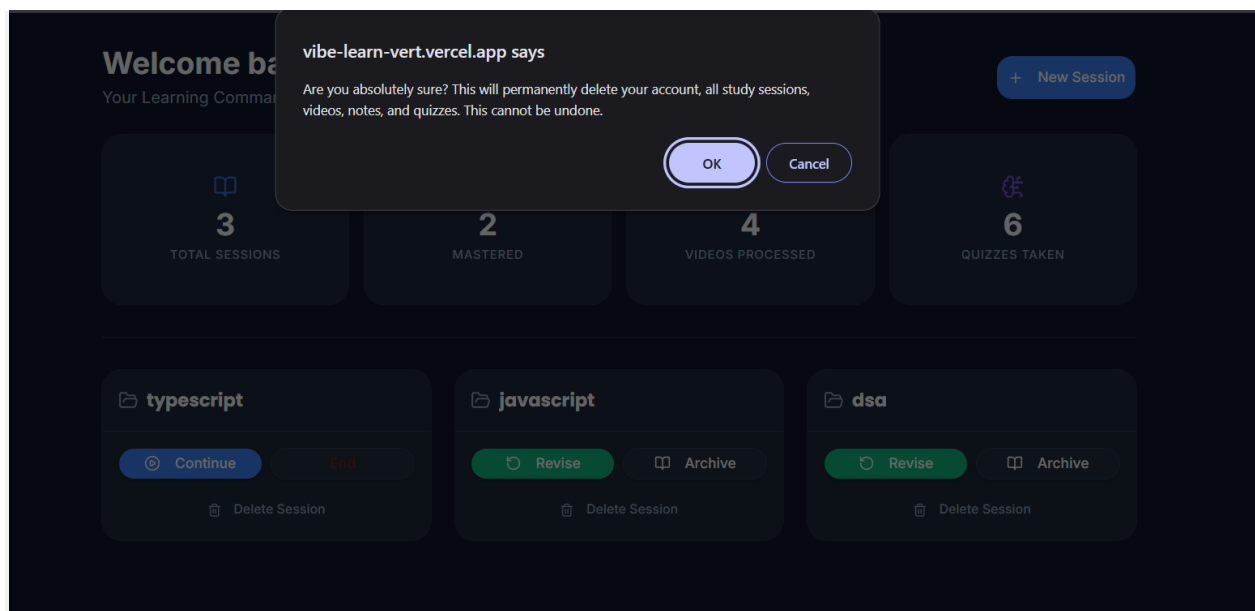
Figure 20.3
Delete Session Confirmation Dialog



20.4 Delete Account

Account deletion, described at the implementation level in Chapter 18.6, is similarly gated behind a confirmation dialog that requires the user to acknowledge the action is irreversible. Because the cascade in this case originates from `auth.users(id)` and every owned table (`sessions`, `quiz_attempts`) references it with `ON DELETE CASCADE`, deleting the `auth` user row is sufficient to remove all of that user's owned data in a single operation, without requiring the application to manually orchestrate a multi-table deletion sequence.

Figure 20.4
Delete Account Confirmation Dialog



Chapter 21 – Premium UI System

21.1 Design System Overview

Phase-I's interface used Tailwind utility classes applied ad hoc per component, which was adequate for a prototype but produced visible inconsistency in spacing, colour usage, and card styling once the application grew beyond its original five pages. Phase-II introduces a semantic colour token system layered on top of Tailwind CSS, where component-level classes reference named tokens (background, foreground, card, border, muted, accent, and their dark-mode counterparts) rather than literal colour values, allowing every component to respond correctly to a theme change without per-component dark-mode overrides. The engineering benefit of this approach is maintainability: a future palette change requires editing the token definitions in one place rather than auditing every component for hard-coded colour values, and it guarantees that any newly added component automatically inherits correct light- and dark-mode behaviour simply by using the existing token classes.

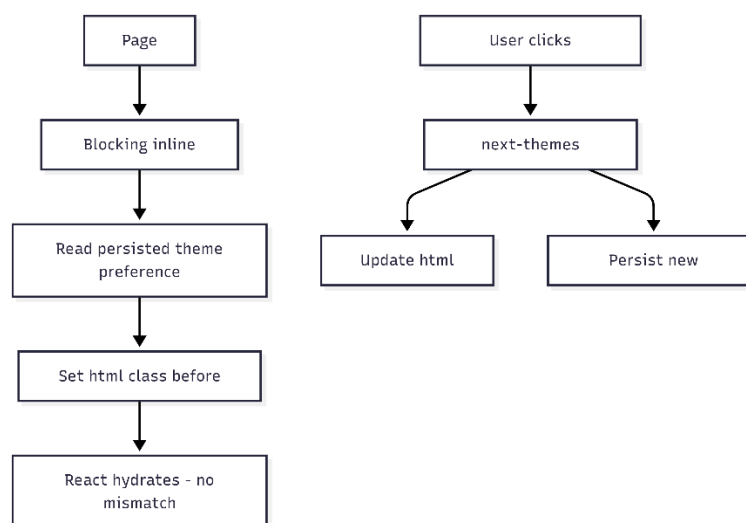
21.2 Typography and Spacing

A consistent type scale and spacing rhythm, derived from Tailwind's default scale but constrained to a deliberately small subset of permitted sizes, was applied across all redesigned pages to reduce the visual inconsistency present in the Phase-I interface, where heading sizes and card padding varied somewhat from page to page as each was built independently during the original sprint.

21.3 Dark Mode and Theme Provider

Dark mode is implemented using next-themes, which wraps the application root and manages the html element's class attribute and a persisted theme preference, while specifically avoiding the flash-of-incorrect-theme problem common to naive implementations by injecting a small blocking script that sets the correct class before first paint. A ThemeToggle component, placed in the sidebar, allows the user to switch between light, dark, and system-preference modes.

Figure 21.1
Theme Provider Architecture



21.4 Sidebar Navigation

The Phase-I pattern of per-page header links was replaced with a persistent, collapsible sidebar (AppSidebar) present on all authenticated routes, containing links to the dashboard and the theme toggle, and collapsing to an icon-only rail on narrower viewports to preserve content width on the session workspace and quiz pages.

Figure 21.2
Expanded Sidebar

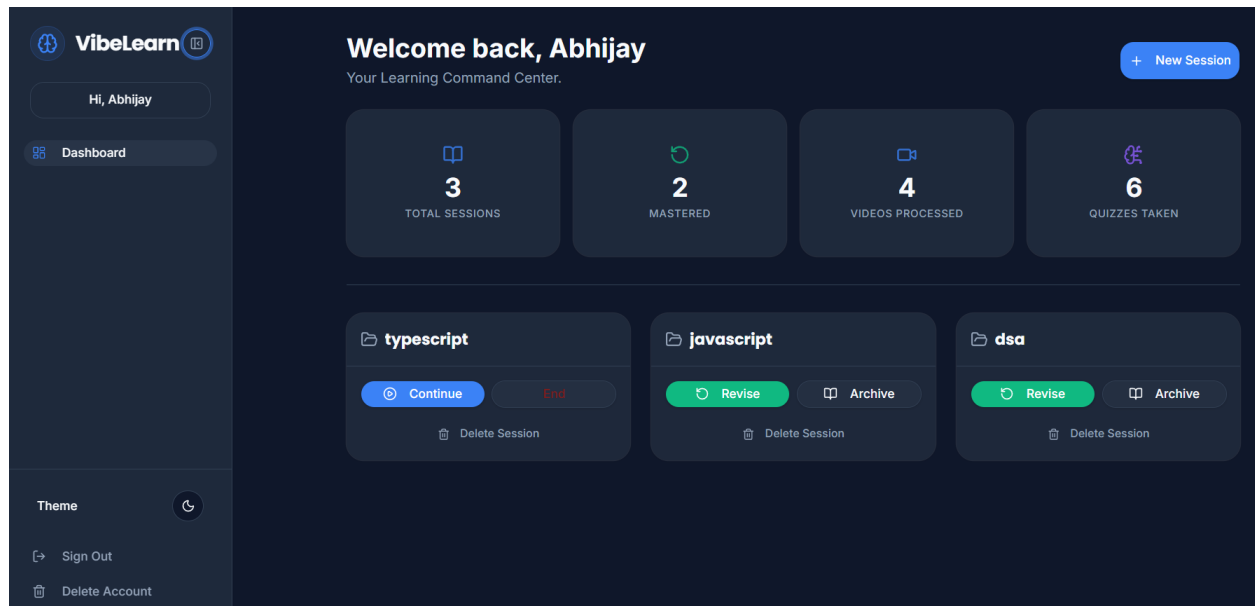
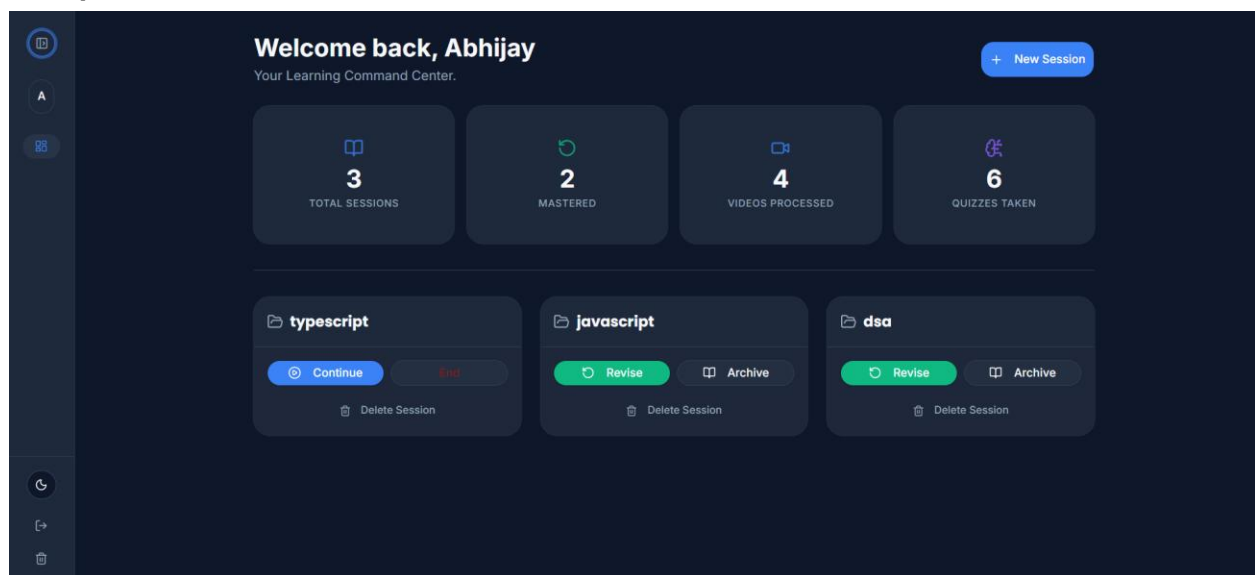


Figure 21.3
Collapsed Sidebar



21.5 Dashboard Redesign

The dashboard was redesigned around a row of analytics cards (described further in Chapter 22) above the existing session grid, with each SessionCard restyled to use the semantic token system and to surface session status more clearly than the Phase-I version.

Figure 21.4

Dashboard Overview — Light Theme

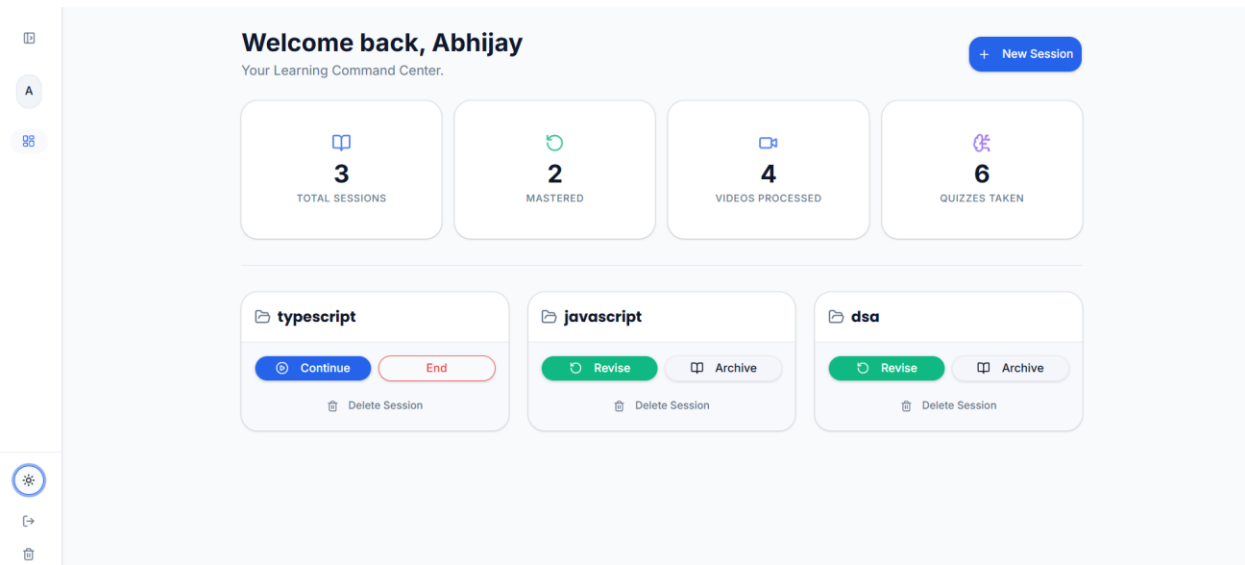
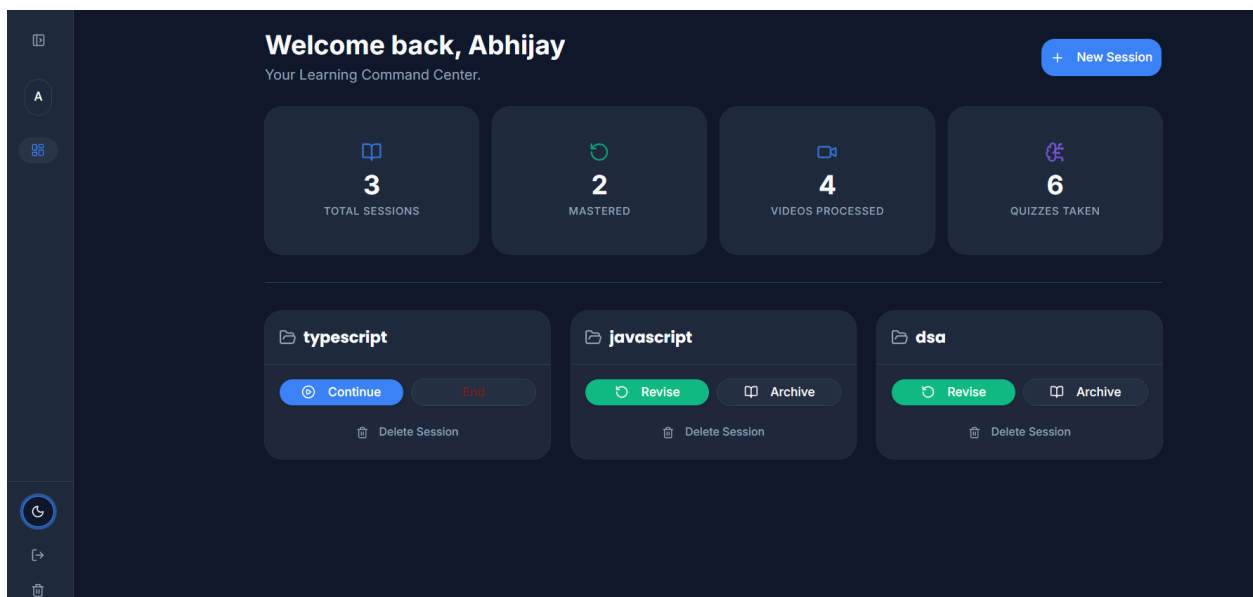


Figure 21.5

Dashboard Overview — Dark Theme

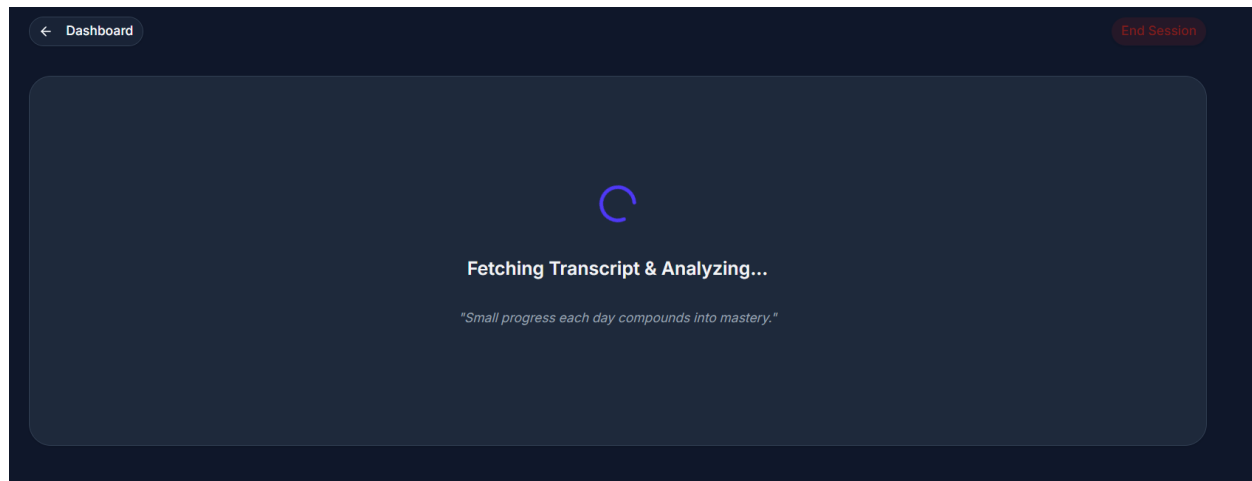


21.6 Loading Experience

A dedicated LoadingScreen component replaced the Phase-I pattern of bare disabled buttons and blank panels during note and quiz generation. It displays a rotating set of short, encouraging or mildly humorous study-related messages while a request is in flight, intended primarily to improve perceived performance during the several-second window of a cache-miss generation request, rather than to convey any technical information.

Figure 21.6

Loading Experience — Notes Generation



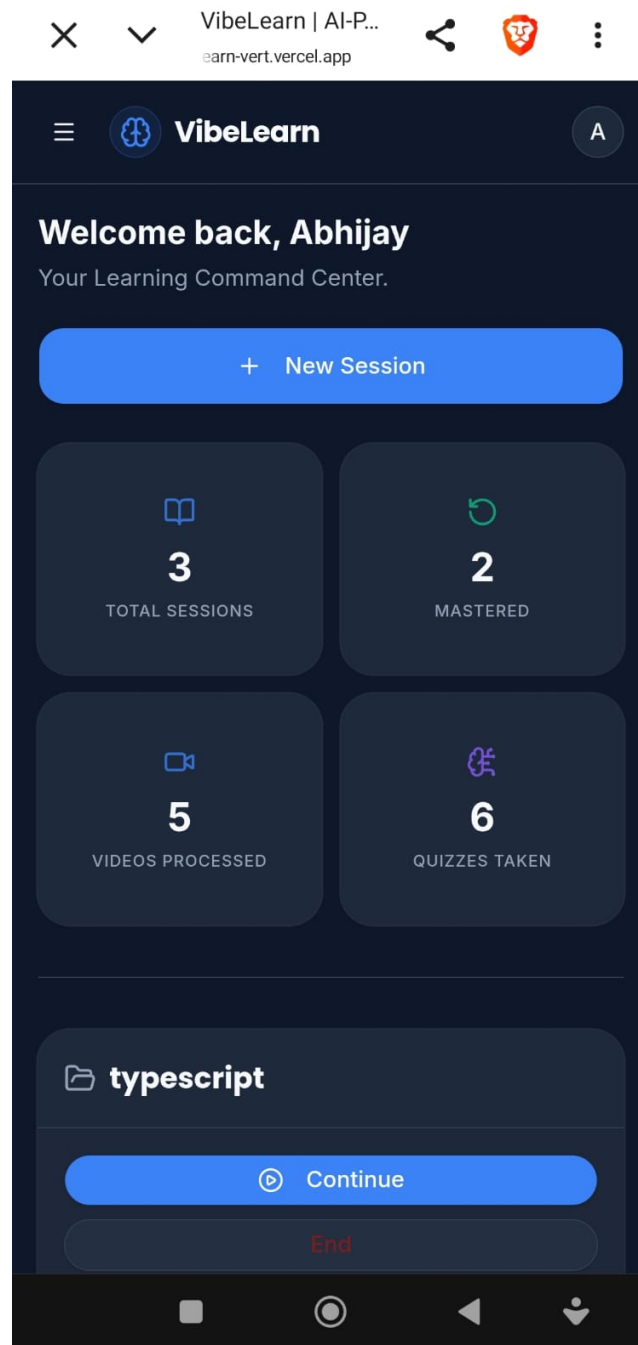
21.7 Quiz, Revision, Landing, and Authentication Page Redesigns

The quiz and revision interfaces were restyled onto the same semantic token system without changing their underlying interaction model, described in Chapters 16 and 17. The landing page was rebuilt around the Google OAuth sign-in button described in Chapter 18, replacing the Phase-I username form entirely. The authentication callback page was kept deliberately minimal, displaying only a brief loading indicator during the code-exchange step described in Chapter 18.2.

21.8 Responsive and Accessibility Improvements

Sidebar collapse behaviour, touch target sizing on the quiz option buttons, and colour contrast between foreground and background semantic tokens in both light and dark modes were each adjusted during manual testing on a narrower viewport to improve usability on mobile-width browsers, though no dedicated mobile application or PWA packaging was undertaken in this phase (see Chapter 27).

Figure 21.7
Mobile Dashboard Layout



Chapter 22 – Analytics Architecture

22.1 Motivation for PostgreSQL RPCs

The redesigned dashboard's analytics cards require aggregated figures — total sessions, total videos processed, average quiz score, and count of identified weak topics — each of which, computed naively, requires scanning and joining across sessions, videos, quizzes, and quiz_attempts for the requesting user. An initial implementation performed this aggregation in the Next.js Route Handler itself, issuing several sequential Supabase JS client queries and reducing the results in JavaScript. This approach is described in full, including the timeout problem it caused, in Chapter 23.6.

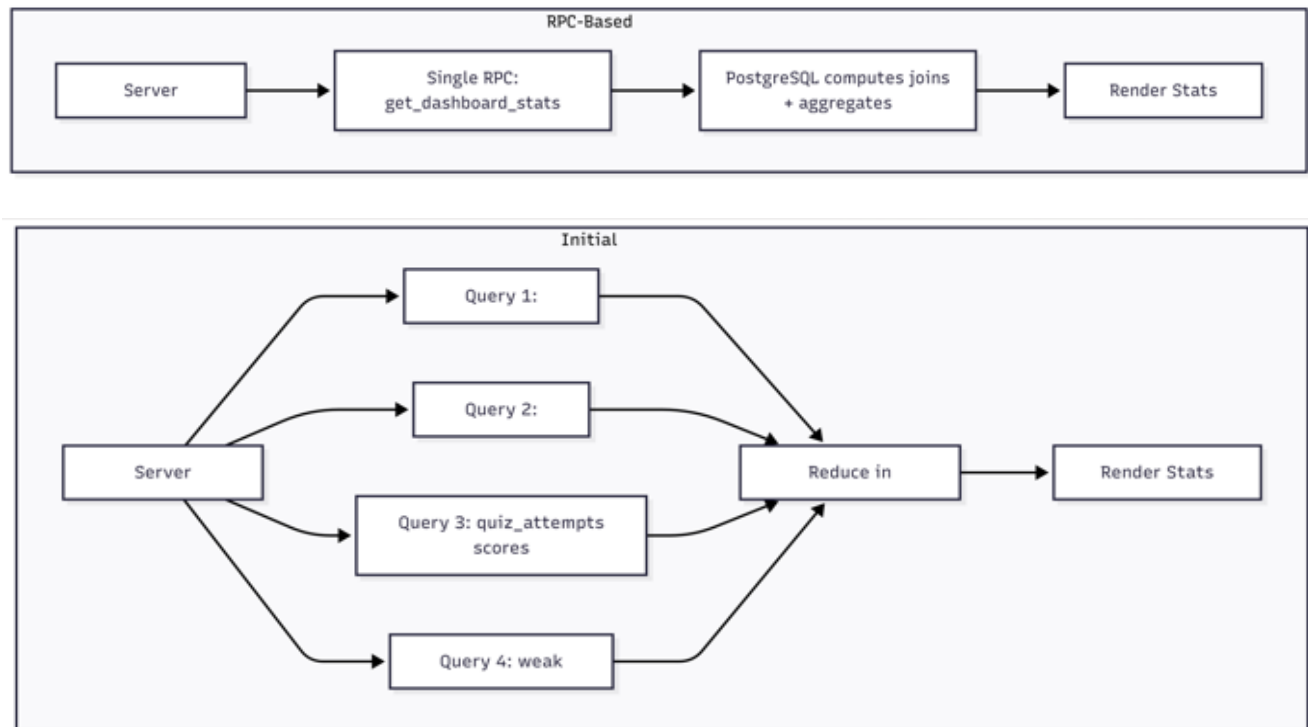
22.2 RPC-Based Aggregation

Dashboard statistics are generated using a PostgreSQL RPC function exposed through Supabase. The application invokes the RPC to retrieve aggregated values such as total sessions, processed videos, average quiz score, and weak-topic count in a single database round trip.

22.3 Why This Avoided Vercel Timeout Limits

Executing the aggregation as a single PostgreSQL function call replaces several sequential network round trips between the Vercel serverless function and the Supabase database with one round trip, and lets PostgreSQL's own query planner handle the joins and aggregations natively rather than pulling rows across the network for in-application reduction. This directly addressed the dashboard analytics timeout problem encountered during testing, discussed in Chapter 23.6, and is the clearest example in this phase of a broader principle the author took away from the work: aggregation belongs in the database, not in the serverless function calling it, whenever the two are this tightly coupled.

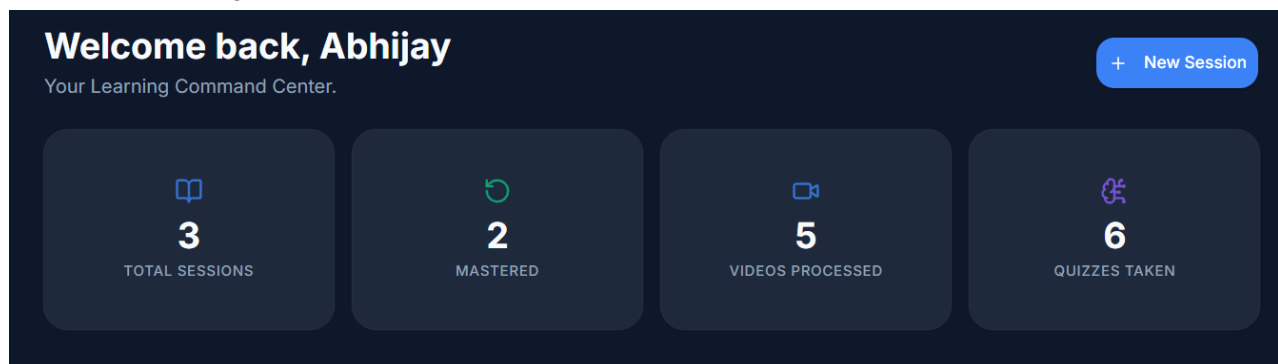
Figure 22.1
Dashboard Analytics RPC Flow



22.4 Dashboard Statistics Display

The RPC's result is rendered as four DashboardStatsCard components above the session grid, giving the user an at-a-glance summary of their study activity that did not exist in any form in Phase-I.

Figure 22.2
Dashboard Analytics Cards



Chapter 23 – Challenges Faced and Engineering Tradeoffs

23.1 Authentication Redirect Flickering

Problem: immediately after migrating the landing page to the Supabase OAuth flow, authenticated users would briefly see the unauthenticated landing page flash before being redirected to the dashboard on every page load. Why it happened: the AuthProvider's session check is inherently asynchronous, and the page's initial render occurred before that check resolved. Investigation: console logging confirmed the redirect logic itself was correct and only the timing of the initial render was at fault. Possible approaches considered: a global loading gate blocking all rendering until the auth check resolves, versus a more surgical per-page loading state. Chosen solution: a single top-level loading gate in the root layout, accepting a brief blank-screen flash in place of the previous flash of incorrect content, on the grounds that a blank screen is less jarring and less of a stated information-disclosure concern than briefly rendering unauthenticated UI to an authenticated user. Lesson learned: client-side auth state should be treated as unavailable-until-proven-otherwise on every render, not merely on the first one.

23.2 Supabase Migration and GoTrue Warnings

During the early migration, console warnings from the Supabase client regarding deprecated session-retrieval methods (specifically warnings against using the locally cached session object for authorization decisions) surfaced repeatedly. Investigation into the Supabase documentation clarified the distinction between `getSession()` (fast, but reads a potentially stale, unverified local token) and `getUser()` (slower, but cryptographically re-verified against the server). The chosen solution, applied consistently across every server-side route as described in Chapter 18.3, was to use `getUser()` exclusively for any authorization decision, reserving `getSession()` only for non-security-critical client-side UI state such as showing or hiding the sign-in button. The tradeoff is a small amount of additional server-side latency per request in exchange for closing a class of session-spoofing risk that the warning was specifically pointing at.

23.3 Duplicate Profiles and Database Race Conditions

Early in the caching work, a race condition surfaced where two near-simultaneous requests for the same new video both passed the cache-lookup check (finding no existing row) before either had inserted its own row, resulting in two competing INSERT statements once the `yt_video_id` UNIQUE constraint was added. Why it happened: the check-then-insert pattern is not atomic without additional database-level protection. Approaches considered: application-level locking, a retry-on-conflict loop, or an UPSERT-style `ON CONFLICT` clause. Chosen solution: rewriting the insert as an `INSERT ... ON CONFLICT (yt_video_id) DO UPDATE` statement, so that the second of two racing requests updates (effectively a no-op refresh) rather than fails, and both requests can safely read back a single, consistent row afterward. Lesson learned: any check-then-act pattern against a uniqueness constraint under concurrent access needs either a transaction-level lock or an idempotent upsert; a plain SELECT followed by a plain INSERT is not safe.

23.4 Why SQL Constraints Initially Failed to Apply, and Why Old Duplicate Rows Blocked Migration

When the UNIQUE constraint on `videos.yt_video_id` was first added via a migration, it failed outright with a constraint-violation error rather than applying successfully. Investigation revealed that the existing Phase-I data already contained multiple videos rows sharing the same `yt_video_id`, a direct artefact of the very per-user duplication problem the cache was being built to solve. The chosen solution was a one-off data-cleanup migration that, for each duplicated `yt_video_id`, retained the earliest-created row and re-pointed any quizzes or quiz_attempts referencing the duplicate rows to the retained row before deleting the duplicates, after which the UNIQUE constraint applied cleanly. This was treated as a one-time, irreversible migration step run manually rather than as application code, since it needed to run exactly once against the existing dataset.

23.5 Repeated Transcript and Quiz Generation Prior to Caching

Before the caching layer existed, informal observation during development — the author repeatedly revisiting the same handful of test videos while debugging unrelated features — made directly visible how much redundant Groq token usage and transcript-fetch traffic the Phase-I design generated even from a single developer's own testing activity, which was itself a significant motivating data point behind prioritising Chapter 19's work as early as it was in the Phase-II sprint.

23.6 Dashboard Analytics Timeout and the PostgreSQL RPC Solution

The first implementation of the dashboard statistics cards, described in Chapter 22.1, issued four to five sequential Supabase JS client queries from within the dashboard's server component. Under realistic test data volumes this occasionally approached, and in one observed case exceeded, Vercel's default serverless function execution time limit, producing an intermittent failed dashboard load. The root cause was the combination of network round-trip latency multiplied across several sequential queries plus client-side reduction of joined data that PostgreSQL itself could have performed natively. The chosen solution, detailed in Chapter 22.2, was to replace the sequential client queries with a single RPC call to a PL/pgSQL function, collapsing several round trips into one and moving the join and aggregation work into the database engine itself.

23.7 Theme Provider Hydration and Tailwind Version Issues

Introducing next-themes surfaced a hydration mismatch warning from React, caused by the server rendering the page without knowledge of the client's stored theme preference while the client then immediately applied a different class on mount. This was resolved by following next-themes' documented `suppressHydrationWarning` pattern on the `html` element, which is the library's intended mechanism for this specific, expected mismatch. Separately, introducing additional Tailwind CSS utility classes for the semantic token system surfaced a version-compatibility issue between a newly added dependency expecting Tailwind v4 conventions and the project's existing Tailwind v3 configuration; this was resolved by pinning the dependency to a v3-compatible release rather than undertaking a full Tailwind v4 migration mid-phase, which was judged out of scope for this sprint.

23.8 Sidebar Responsiveness

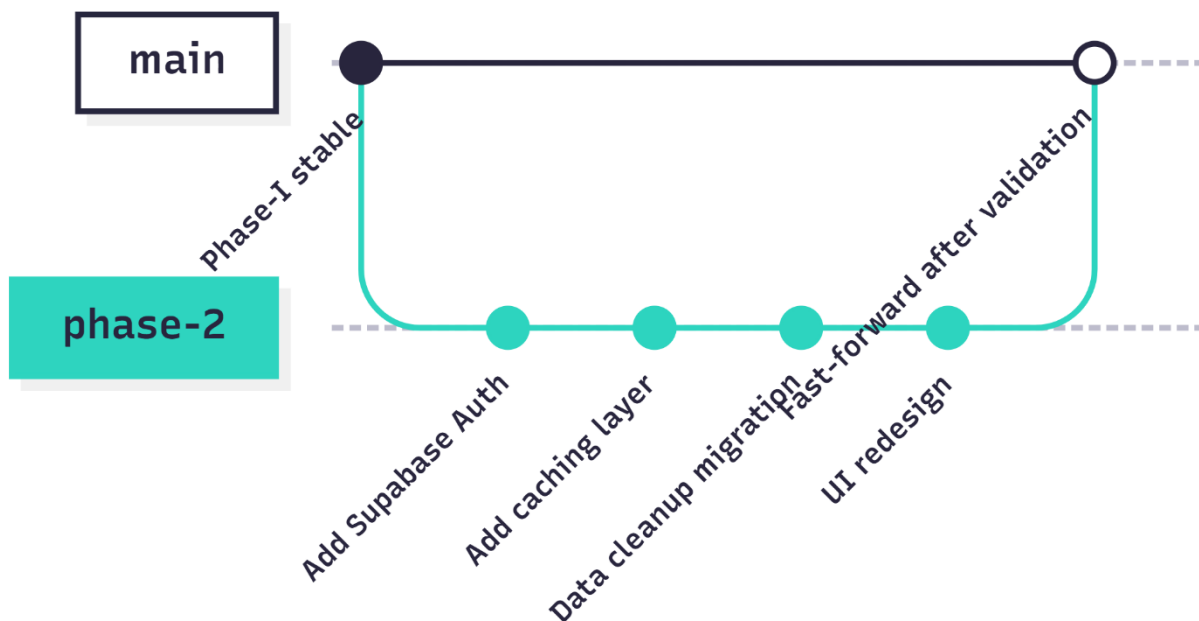
The initial sidebar implementation did not collapse correctly on narrower viewports, overlapping the session workspace's video player. This was resolved by adding an explicit breakpoint-driven collapse state, defaulting to a collapsed icon-only rail below a defined viewport width rather than relying solely on user-toggled collapse.

23.9 Deployment Branch Management

Because the author wished to keep the working Phase-I MVP available and deployable throughout Phase-II development, all authentication and caching work was developed on a dedicated feature branch rather than directly on main, with main only being fast-forwarded once the migration, including the data-cleanup step described in Chapter 23.4, had been validated against a copy of the production database. This is noted here as a process decision rather than a purely technical one, but is included because it materially shaped how several of the riskier migration steps above were sequenced and tested.

Figure 23.1

Deployment Branch Strategy



23.10 Navigation Bugs and Loading Experience Redesign

As referenced in Chapter 16.3, quiz exit and finish navigation initially routed users to the dashboard regardless of which session they had started from, losing their place. This was fixed by passing the originating session_id through the quiz route's query parameters and routing both exit and finish actions back to /session/[id] rather than to a hardcoded dashboard route. The loading-experience redesign described in Chapter 21.6 was undertaken partly in response to user testing (the author's own repeated use) revealing that the blank-panel loading state during a cache-miss generation request felt considerably longer than its actual duration.

23.11 Engineering Tradeoffs Analysis

Decision	Chosen Approach	Advantage	Disadvantage
Authentication	Supabase Auth + Google OAuth	Cryptographic identity verification, no custom token code	External provider dependency
Session verification	getUser() on every request	Always re-verified, closes spoofing risk	Added per-request latency vs. local session read
Caching scope	Global cache for notes and video_quiz only	Maximises reuse where safe	Excludes the highest-value personalised content (revision quizzes) by design
Cache key	yt_video_id, not local UUID	Correctly identifies shared content across users	Required denormalising yt_video_id onto quizzes
Cache invalidation	None implemented	Simplicity; avoids a hard distributed-cache problem	Stale content risk in the rare case of a changed video
Authorization model	Application-layer checks, no RLS	Faster to implement consistently across handlers	No database-level defence-in-depth if a handler check is ever missed
Dashboard aggregation	PostgreSQL RPC	Single round trip, avoids serverless timeout	Aggregation logic now lives in SQL, less familiar to debug than TypeScript
UI theming	next-themes + semantic tokens	No flash-of-incorrect-theme, consistent styling	Required restyling every existing component
Branch strategy	Feature branch, main kept stable	Phase-I remained deployable throughout Phase-II	Periodic merge/rebase overhead for a sole developer

Table 11: Engineering Tradeoffs Analysis, Version 2.0

Chapter 24 – Testing Strategy and Test Cases

24.1 Testing Approach (Updated)

The manual, exploratory testing approach established in Phase-I continues in Version 2.0, now additionally covering authentication, ownership, and caching paths. As in Phase-I, automated unit and integration testing was considered and again deferred, on the same reasoning that tooling overhead was judged disproportionate to the value for a sole-developer prototype at this stage; this remains noted as a Phase-III candidate in Chapter 27.

24.2 Updated Test Cases

TC	Description	Input	Expected Output	Result
TC13	Google OAuth sign-in	User completes Google consent screen	Session established, redirected to dashboard	Pass
TC14	Unauthenticated API access	Request to /api/generate-notes with no session	401 Unauthorized response	Pass
TC15	Cross-user session access	User A requests User B's session_id	403 Forbidden response	Pass
TC16	Notes cache hit	Second user submits an already-cached yt_video_id	Notes returned without Groq call, sub-second response	Pass
TC17	Notes cache miss	First-time yt_video_id submitted	Full generation pipeline executes, row inserted	Pass
TC18	Quiz cache hit	Second user requests quiz for cached video	Existing quiz returned without Groq call	Pass
TC19	Revision quiz never cached	Two users with different weak topics request revision	Distinct, personalised quizzes generated for each	Pass
TC20	Delete session	Confirm deletion of an owned session	Session and dependent rows removed; cached video retained if referenced elsewhere	Pass
TC21	Delete account	Confirm account deletion	All owned sessions, quizzes, and attempts removed	Pass
TC22	Dark mode toggle	User toggles theme	Theme persists across reload, no flash of incorrect theme	Pass
TC23	Dashboard stats load	User with existing history opens dashboard	Four stats cards populated via single RPC call	Pass
TC24	Notes Only / Video Only modes	User toggles either mode	Mutually exclusive panel visibility enforced	Pass

Table 12: Test Cases — Phase-II Additions (TC01–TC12 from Phase-I remain valid and continue to pass)

Chapter 25 – Results and Discussion

25.1 Feature Completeness

All primary and secondary objectives defined in Chapter 4 were implemented and verified through the test cases in Chapter 24. The platform now supports the complete Phase-I learning loop under verified authentication, with global caching active for notes and per-video quizzes and dynamic personalisation preserved for revision quizzes.

25.2 Security

The identity-spoofing vulnerability identified as the most significant Phase-I limitation has been structurally eliminated through the authentication architecture described in Chapter 18 and the ownership verification described in Chapter 20. The absence of database-level row-level security remains a residual gap, noted explicitly in Chapter 26, but every currently existing access path is mediated by an application-layer check.

25.3 Performance and API Efficiency

No formal benchmarking infrastructure was built in this phase, consistent with the qualitative-observation approach used in Phase-I. Informally, cached note and quiz retrieval was consistently and noticeably faster than fresh generation during manual testing, dropping from a multi-second LLM-bound request to a sub-second database read, and repeated study of overlapping test videos during development no longer produced repeated Groq API calls, which was directly observable in the absence of repeated console-logged generation events for videos already present in the cache. These improvements are reported qualitatively and intentionally avoid implying precision that informal, single-developer observation cannot support.

25.4 Scalability

The move of dashboard aggregation into a PostgreSQL RPC, described in Chapter 22, removed the clearest scalability risk identified during this phase — the intermittent serverless timeout under sequential client-side queries — without requiring any change to the serverless deployment model itself.

25.5 User Experience

The interface redesign described in Chapter 21, combined with the improved loading experience and corrected navigation flows, produced a noticeably more polished day-to-day experience during the author's own continued use of the platform for personal study, compared to the functional but visually inconsistent Phase-I interface.

25.6 Maintainability

Centralising the authentication preamble and adopting a consistent ownership-check pattern across route handlers, as described in Chapter 14, made the codebase easier to extend confidently: new routes added late in this phase (delete-session, delete-account) were written quickly by following the established pattern rather than by re-deriving authorization logic from scratch.

Chapter 26 – Limitations

26.1 Known Limitations (Version 2.0)

- No database-level row-level security: authorization is enforced entirely in application code. A future bug in a route handler's ownership check would not be caught by any database-level defence-in-depth.
- No cache invalidation: cached notes and quizzes are assumed permanent once generated, with no mechanism to refresh or expire them if underlying video content changes.
- Language restriction unchanged from Phase-I: only English-language transcripts and prompts are supported.
- No automated test suite: testing remains manual and exploratory, as in Phase-I, which increases regression risk as the codebase grows.
- Single OAuth provider: Google is the only sign-in method; users without a Google account cannot use the platform.
- No offline functionality: the platform continues to require active internet connectivity for all operations.
- Adaptive algorithm simplicity unchanged: weak topic aggregation remains an unweighted Set-based deduplication, not a statistical model.

Chapter 27 – Future Scope

27.1 Version 3.0 Development Priorities

With authentication, caching, and the core interface redesign now complete, the following genuinely new directions are proposed for a future Version 3.0, none of which were implemented in this phase:

Feature	Description	Priority
Spaced Repetition Scheduling	Implement an SM-2 or FSRS-based scheduler to determine optimal review timing per topic	High
Row-Level Security	Add PostgreSQL RLS policies as defence-in-depth beneath the existing application-layer checks	High
AI Flashcard Generation	Generate Anki-compatible flashcard decks from session notes	Medium
Export Notes	Allow notes to be exported as Markdown or PDF for offline reference	Medium
Advanced Analytics	Visual trend charts of quiz performance over time, building on the Chapter 22 RPC foundation	Medium
Collaborative Learning	Allow multiple users to contribute videos to a shared session	Low
PWA Support / Offline Mode	Package the application as an installable Progressive Web App with limited offline access to cached notes	Low
AI Tutoring	A conversational follow-up interface allowing users to ask clarifying questions against their session notes	Low
Mobile Application	A React Native or equivalent mobile client	Low
Multi-language Support	Process transcripts and prompts in languages other than English	Low

Table 13: Future Scope Prioritisation, Version 3.0 Candidates

Chapter 28 – Conclusion

Version 2.0 of VibeLearn directly addresses the most significant limitation identified at the end of Phase-I: the absence of verifiable user identity. By migrating to Supabase Authentication with Google OAuth and enforcing server-side session verification on every protected route, the platform closes the identity-spoofing vulnerability that the Phase-I report explicitly flagged as unacceptable for anything beyond a trusted-user prototype.

That authenticated foundation, in turn, made possible an optimisation that was not available in Phase-I: a global, content-addressed cache for AI-generated notes and per-video quizzes, which eliminates redundant transcript extraction and redundant LLM inference whenever more than one user studies the same YouTube video, while deliberately preserving full per-user personalisation for the adaptive revision engine, where caching would have been actively harmful to the feature's purpose.

Beyond these two structural changes, Phase-II delivered a comprehensive interface redesign — a semantic, theme-aware design system, dark mode, sidebar navigation, and redesigned core pages — and a database-side analytics architecture built on PostgreSQL RPC functions that resolved a real serverless timeout risk encountered during development. Each of these changes was driven by a concrete problem encountered through the author's own continued use of the platform, rather than by speculative feature addition, consistent with the engineering discipline established in Phase-I.

Taken together, these improvements transform VibeLearn from a functionally complete but security- and consistency-limited MVP into a substantially more production-ready platform, while preserving the original AI-powered learning workflow — transcript extraction, note generation, quiz assessment, and adaptive revision — entirely unchanged from the user's perspective. The remaining limitations documented in Chapter 26, most notably the absence of database-level row-level security and of any cache invalidation mechanism, are well understood and define a clear starting point for a future Version 3.0, prioritised in Chapter 27.

As with Phase-I, the value of this phase to the author extended beyond the application itself: production authentication flows, PostgreSQL constraint and migration handling under real data, and database-side aggregation via RPC functions were each learned through the friction of solving an actual problem rather than through tutorial exercise, and each is directly relevant to the Software Development Engineer and data-adjacent roles the author is targeting in the upcoming on-campus internship season.

References

- [1] Ebbinghaus, H. (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie. Duncker & Humblot.
- [2] Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. arXiv:2005.14165.
- [3] Elkins, S., et al. (2023). How Teachers Can Use Large Language Models and Bloom's Taxonomy to Create Educational Quizzes. arXiv:2301.05914.
- [4] Bunt, A., Lount, M., & Lauzon, C. (2014). Are Explanations Always Important? A Study of Deployed, Low-Cost Intelligent Tutoring Systems. ITS 2014.
- [5] Sleeman, D., & Brown, J. S. (1982). Intelligent Tutoring Systems. Academic Press, London.
- [6] Next.js Documentation. (2024). App Router. <https://nextjs.org/docs/app>
- [7] Supabase Documentation. (2024). Auth. <https://supabase.com/docs/guides/auth>
- [8] Supabase Documentation. (2024). JavaScript Client Library. <https://supabase.com/docs/reference/javascript>
- [9] Groq Documentation. (2024). API Reference. <https://console.groq.com/docs>
- [10] Meta AI. (2024). Llama 3.3 Model Card. <https://llama.meta.com>
- [11] PostgreSQL Documentation. (2024). PL/pgSQL — SQL Procedural Language. <https://www.postgresql.org/docs/current/plpgsql.html>
- [12] OWASP Foundation. (2024). Authentication Cheat Sheet. <https://cheatsheetseries.owasp.org>
- [13] next-themes Documentation. (2024). <https://github.com/pacocoursey/next-themes>
- [14] Shadcn/UI Documentation. (2024). <https://ui.shadcn.com>
- [15] Tailwind CSS Documentation. (2024). <https://tailwindcss.com/docs>

Appendix

Appendix A: Project Statistics (Version 2.0)

Metric	Value
Total Pages / Routes	7 (including OAuth callback route)
API Routes	7 (3 generation + 2 deletion + auth callback + RPC-backed dashboard read)
Database Tables	6 application tables (users, profiles, sessions, videos, quizzes, and quiz_attempts) with authentication managed by Supabase Auth (auth.users).
New Chapters vs. Phase-I	5 (Authentication, Caching, Ownership, UI System, Analytics)
Major New Technologies	Supabase Authentication, next-themes, Supadata, Lucide React
PostgreSQL RPC Functions	1 (dashboard statistics aggregation)
Development Duration	Phase-II, approximately 3–4 additional weeks
Language	TypeScript
Runtime	Node.js (Next.js), PL/pgSQL (Supabase)

Table A1: Project Statistics Summary, Version 2.0

Appendix B: Environment Variables (Updated)

```
# .env.local

GROQ_API_KEY=<groq_api_key>

NEXT_PUBLIC_SUPABASE_URL=<supabase_project_url>

NEXT_PUBLIC_SUPABASE_ANON_KEY=<supabase_anon_key>

SUPABASE_SERVICE_ROLE_KEY=<supabase_service_role_key>

SUPADATA_API_KEY=<supadata_api_key>

NEXT_PUBLIC_SITE_URL=<deployed_site_url_for_OAuth_redirect>
```

Appendix C: Directory Structure (Updated)

```
vibelearn/ |-- app/ |    |-- api/ |    |    |-- generate-notes/route.ts |    |    |-- generate-quiz/route.ts |    |    |-- generate-master-quiz/route.ts |    |    |-- delete-session/route.ts |    |    |-- delete-account/route.ts |    |-- auth/callback/route.ts |    |-- dashboard/page.tsx |    |-- session/[id]/page.tsx |    |-- quiz/[id]/page.tsx |    |-- revision/[id]/page.tsx |    |-- layout.tsx |    |-- page.tsx |-- components/ |    |-- VideoCard.tsx |    |-- QuizQuestion.tsx |    |-- ScoreDisplay.tsx |    |-- SessionCard.tsx |    |-- MarkdownRenderer.tsx |    |-- AppSidebar.tsx |    |-- ThemeToggle.tsx |    |-- LoadingScreen.tsx |    |-- DeleteConfirmDialog.tsx |    |-- DashboardStatsCard.tsx |-- context/ |    |-- AuthProvider.tsx |-- lib/ |    |-- supabase-server.ts |    |-- supabase-client.ts |    |-- groq.ts |-- types/ |    |-- index.ts |-- .env.local |-- next.config.ts |-- tailwind.config.ts |-- package.json
```

Appendix D: Figure Index

All figures in this report are numbered by chapter (e.g., Figure 18.1 refers to the first figure in Chapter 18). Screenshot placeholders are marked [Insert Screenshot Here] and Mermaid diagram placeholders are marked [Insert Mermaid Diagram Here], each accompanied by the corresponding Mermaid source code for later rendering.

Appendix E: Table Index

Table 1: Version 1.0 to Version 2.0 Comparison Overview | Table 2: Comparison of Existing Learning Platforms | Table 3–4: Functional and Non-Functional Requirements | Table 5: Database Schema Evolution | Table 6: Technology Stack Summary | Table 7: Application Routes | Table 8: API Route Summary | Table 9: Security Comparison | Table 10: Caching Architecture Summary | Table 11: Engineering Tradeoffs Analysis | Table 12: Test Cases | Table 13: Future Scope Prioritisation | Table A1: Project Statistics Summary.